

《现代C++编程实现》电子版: <https://leanpub.com/c01>

8. 类和对象

YouTube频道: [hwdong](https://www.youtube.com/hwdong)

博客: hwdong-net.github.io

B站: [hw-dong](https://www.bilibili.com/video/av1000000000)

8.1 过程式编程

- 过程式编程是一种以**过程**（函数或子程序）为中心的编程范式。它的核心思想是将程序分解为一系列有序的步骤，每个步骤通过函数或子程序来实现。
- 在过程式编程中，数据和操作是分离的，数据通常存储在全局变量或结构体中，而操作则通过函数来实现。

智能家居系统

智能家居系统应具备以下基本功能：

- 用户可以通过命令行输入指令，控制灯光、空调等设备。
- 系统能够根据温度传感器的数据，自动调节空调温度。
- 用户可以设置预设的场景模式，例如“回家模式”和“离家模式”。
- 系统能够执行定时任务，例如定时开关灯。

智能家居系统：过程式分析与设计

1. 需求分析
2. 系统分解与模块划分
3. 数据结构设计
4. 函数设计
5. 系统流程设计
6. 编码实现（本节重点在于设计，编码仅作示例）

智能家居系统：需求分析

- 需求分析是开发的第一步，旨在明确系统目标和功能。智能家居系统需通过科技手段控制灯光、空调等设备，提升生活便利性和舒适性。

根据题目要求，系统需具备以下功能：

- **用户控制**：通过命令行输入指令，控制灯光、空调等设备（如“light_on”、“light_off”）。
- **自动调节**：根据温度传感器数据，自动调节空调温度（温度 $> 28^{\circ}\text{C}$ 时设为 22°C ，温度 $< 20^{\circ}\text{C}$ 时设为 26°C ）。
- **场景模式**：支持“回家模式”（打开灯光、调节空调）和“离家模式”（关闭所有设备）。
- **定时任务**：定时开关灯（如每小时切换灯光状态）。

智能家居系统：需求分析

• 用例分析

- 用例1：用户控制灯光
 - **描述：**用户输入“light_on”，系统打开灯光并提示成功。
 - **系统动作：**
 1. 接收“light_on”指令。
 2. 执行灯光打开操作。
 3. 向用户反馈“Light is ON”。

智能家居系统：需求分析

• 用例分析

• 用例2：回家模式

- **描述：**用户输入“home”，系统打开灯光、调节空调至合适温度，提示“Home mode activated”。
- **系统动作：**
 1. 接收“home”指令。
 2. 激活“回家模式”场景。
 3. 执行灯光打开和空调调节操作。
 4. 反馈“Home mode activated”。

智能家居系统：需求分析

• 用例分析

- 用例3：离家模式

- **描述：**用户输入“away”，系统关闭所有设备，提示“Away mode activated”。
- **系统动作：**
 1. 接收“away”指令。
 2. 激活“离家模式”场景。
 3. 关闭灯光和空调。
 4. 反馈“Away mode activated”。

智能家居系统：需求分析

• 用例分析

- 用例4：温度自动调节

- **描述：**系统周期性检查温度，若 $> 28^{\circ}\text{C}$ ，空调设为 22°C 并开启；若 $< 20^{\circ}\text{C}$ ，设为 26°C 并开启；否则关闭空调。
- 系统动作：
 1. 获取温度传感器数据。
 2. 判断温度阈值。
 3. 控制空调状态和温度。

智能家居系统：需求分析

• 用例分析

• 用例5：定时开关灯

- **描述：**系统每小时自动切换灯光状态（开→关或关→开）。
- **系统动作：**
 1. 周期性检查定时器。
 2. 若到达时间阈值，切换灯光状态。

智能家居系统：系统分解与模块划分

根据需求，系统分解为以下模块：

- **指令处理模块：**接收并解析用户指令（如“light_on”、“home”），调用相应功能。
- **设备控制模块：**控制设备状态（如灯光开关、空调温度）。
- **场景管理模块：**管理“回家模式”和“离家模式”，批量执行设备控制。
- **传感器数据采集模块：**周期性获取温度数据。
- **自动化控制模块：**根据温度数据自动调节空调。
- **定时任务模块：**管理定时开关灯任务。
- **反馈模块：**向用户发送操作结果提示。

智能家居系统：数据结构设计

- 数据结构用于存储设备状态、场景信息和传感器数据。过程式编程通常使用结构体（struct）组织数据，

智能家居系统：数据结构设计

- 设备信息：

```
typedef enum { LIGHT, AIR_CONDITIONER } DeviceType;

typedef struct {
    int id;           // 设备ID
    char name[50];    // 设备名称
    DeviceType type;  // 设备类型
    int status;       // 0: 关, 1: 开
    int value;        // 属性值（如空调温度）
} DeviceInfo;

DeviceInfo devices[MAX_DEVICES];
int deviceCount = 0;
```

智能家居系统：数据结构设计

- 场景信息：

```
typedef struct {  
    int deviceId;    // 目标设备ID  
    char action[20]; // 动作（如 "turnOn", "setTemperature"）  
    int value;       // 动作值（如温度）  
} Command;  
  
typedef struct {  
    int id;           // 场景ID  
    char name[50];    // 场景名称（如 "home"）  
    Command commands[MAX_COMMANDS]; // 命令列表  
    int commandCount; // 命令数量  
} SceneInfo;  
  
SceneInfo scenes[MAX_SCENES];  
int sceneCount = 0;
```

智能家居系统：数据结构设计

- 传感器数据：

```
typedef struct {  
    float temperature; // 温度值  
} SensorData;
```

智能家居系统：函数设计

- 指令处理模块：
 - `int processCommand(char* command):`
 - **功能：**解析用户指令，调用场景管理或设备控制函数。
 - **参数：**`command`（如“light_on”）。
 - **返回值：**1（成功），0（失败）。
 - **示例：**接收“home”，调用 `activateScene("home")`。

智能家居系统：函数设计

- 场景管理模块：
 - `int activateScene(char* sceneName):`
 - **功能：**根据场景名称查找并执行场景命令。
 - **参数：**sceneName (如“home”)。
 - **返回值：**场景ID (成功)，-1 (失败)。
 - **实现：**遍历 scenes 数组，找到匹配场景，执行其 commands。

智能家居系统：函数设计

- 设备控制模块：
 - `int controlDevice(int deviceId, char* action, int value):`
 - **功能：**根据设备ID和动作控制设备。
 - **参数：**deviceId（设备ID），action（如“turnOn”），value（如温度）。
 - **返回值：**1（成功），0（失败）。
 - **实现：**根据 action 更新 devices 数组中的设备状态。
 - `int turnOnDevice(int deviceId):` 打开设备。
 - `int turnOffDevice(int deviceId):` 关闭设备。
 - `int setDeviceValue(int deviceId, int value):` 设置设备属性（如温度）。

智能家居系统：函数设计

- 传感器数据采集模块：
 - `float getTemperatureData()`:
 - **功能：** 获取温度传感器数据。
 - **返回值：** 当前温度。

智能家居系统：函数设计

- 自动化控制模块：
 - void adjustTemperature(float temperature):
 - **功能：**根据温度调节空调。
 - **实现：**若温度 > 28°C，调用 controlDevice(airConditionerId, "setTemperature", 22) 并打开空调。

智能家居系统：函数设计

- 定时任务模块：
 - void checkTimer():
 - **功能：**检查定时器，切换灯光状态。
 - **实现：**每小时调用 controlDevice(lightId, lightStatus ? "turnOff" : "turnOn", 0)。

智能家居系统：函数设计

- 反馈模块：
 - `void provideFeedback(char* message):`
 - **功能：**向用户发送提示（如“Home mode activated”）。

智能家居系统：系统流程设计

- 系统流程设计描述函数调用关系和数据流，确保系统按预期执行。

智能家居系统：系统流程设计

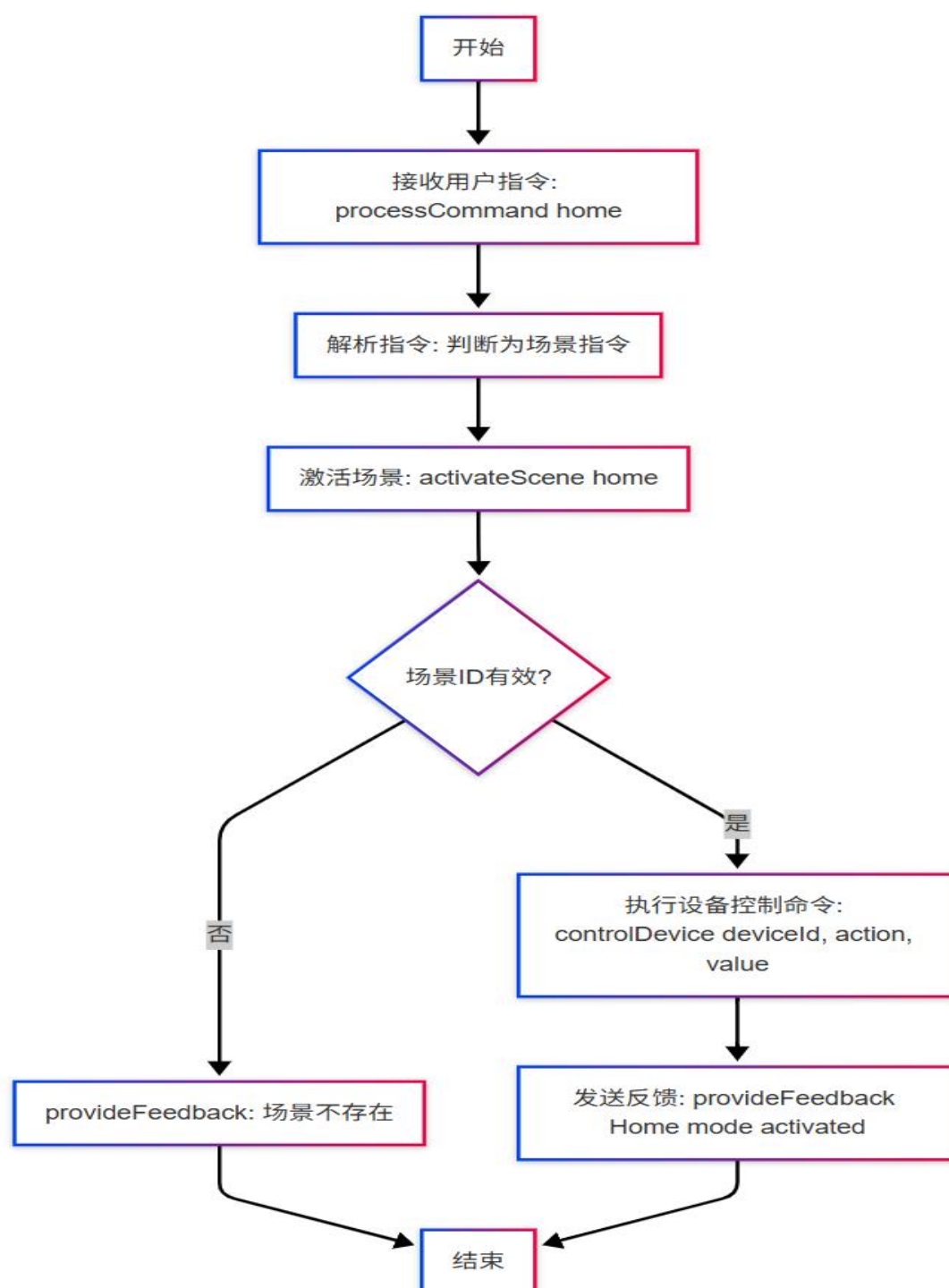
- 示例1：回家模式：

1. `processCommand("home")` 接收指令。
2. 调用 `activateScene("home")`，查找“home”场景。
3. 遍历场景的`commands`，依次调用`controlDevice(deviceId, action, value)`:
 - 打开灯光： `controlDevice(lightId, "turnOn", 0)`。
 - 调节空调： `controlDevice(acId, "setTemperature", 22)`。
4. 调用 `provideFeedback("Home mode activated")`。

智能家居系统：系统流程设计

“回家模式”的函数调用关系如下：

```
processCommand("home")  
  └─> 判断指令类型  
      └─> activateScene("home")  
          └─> 检查场景ID是否有效  
              └─> if valid:  
                  └─> controlDevice(deviceId, action, value) // 多个设备  
              else:  
                  └─> provideFeedback("场景不存在")  
  └─> provideFeedback("Home mode activated") // 最后反馈用户
```



智能家居系统：系统流程设计

- 示例2：温度自动调节：

1. `getTemperatureData()` 获取温度。

2. `adjustTemperature(temperature)`

判断温度：

- 若 $> 28^{\circ}\text{C}$ ，调用 `controlDevice(acId, "setTemperature", 22)` 和 `controlDevice(acId, "turnOn", 0)`。
- 若 $< 20^{\circ}\text{C}$ ，调用 `controlDevice(acId, "setTemperature", 26)` 和 `controlDevice(acId, "turnOn", 0)`。
- 否则，调用 `controlDevice(acId, "turnOff", 0)`。

```
DeviceInfo devices[MAX_DEVICES];  
int deviceCount = 0;  
SceneInfo scenes[MAX_SCENES];  
int sceneCount = 0;  
float currentTemperature = 30.0;  
int timer = 0;
```



```
int processCommand(char* command) {  
    if (strcmp(command, "light_on") == 0) {  
        controlDevice(1, "turnOn", 0);  
        provideFeedback("Light is ON");  
    } else if (strcmp(command, "home") == 0) {  
        activateScene("home");  
        provideFeedback("Home mode activated");  
    }  
    return 1;  
}
```

```
int controlDevice(int deviceId, char* action, int value) {  
    for (int i = 0; i < deviceCount; i++) {  
        if (devices[i].id == deviceId) {  
            if (strcmp(action, "turnOn") == 0) {  
                devices[i].status = 1;  
            } else if (strcmp(action, "setTemperature") == 0) {  
                devices[i].value = value;  
                devices[i].status = 1;  
            }  
            return 1;  
        }  
    }  
    return 0;  
}
```

```
void adjustTemperature(float temperature) {  
    if (temperature > 28) {  
        controlDevice(2, "setTemperature", 22);  
    } else if (temperature < 20) {  
        controlDevice(2, "setTemperature", 26);  
    } else {  
        controlDevice(2, "turnOff", 0);  
    }  
}
```

8.2 面向对象编程

- 面向对象编程（`OOP`）是一种以“**对象**”为中心的编程范式。它将现实世界中的事物抽象成“对象”，**每个对象都具有自己的属性（数据）和行为（方法）。对象之间通过消息传递进行交互，共同完成任务。**
- 与过程式编程不同，面向对象编程更注重数据的封装和抽象，将数据和操作紧密结合在一起。
- 采用面向对象编程（OOP）思想设计智能家居系统，通过类和对象的协作实现模块化、可扩展的系统。

智能家居系统：面向对象分析与设计

1. 需求分析
2. 识别类和对象
3. 定义类的属性和行为
4. 确定类之间的关系
5. 设计对象的交互
6. 系统架构与模块划分
7. 重构与优化设计（省略）
8. 编码实现（本节重点在于设计，仅提供示例代码）

1. 需求分析

- **目标：**明确系统要做什么，包括功能、用户交互和约束条件。

智能家居系统通过科技手段控制灯光、空调等设备，提升生活便利性和舒适性。根据题目要求，系统需具备以下功能：

- **用户控制：**通过命令行输入指令，控制灯光、空调等设备（如“light_on”、“light_off”）。
- **自动调节：**根据温度传感器数据，自动调节空调温度（温度 $> 28^{\circ}\text{C}$ 时设为 22°C ，温度 $< 20^{\circ}\text{C}$ 时设为 26°C ）。
- **场景模式：**支持“回家模式”（打开灯光、调节空调）和“离家模式”（关闭所有设备）。
- **定时任务：**定时开关灯（如每小时切换灯光状态）。

需求分析：用例分析

- 用例分析明确用户与系统的交互，识别功能、参与者和对象，为后续类设计提供依据。

需求分析：用例分析

- 用例1：用户控制灯光
 - **描述：**用户输入“light_on”，系统打开指定灯光并提示“Light is ON”。
 - **参与者：**用户
 - **交互流程：**
 1. 用户输入“light_on”。
 2. 系统解析指令，定位灯光设备。
 3. 系统打开灯光，提示“Light is ON”。
 - **对象：**用户、控制中心、灯光。

需求分析：用例分析

- 用例2：回家模式

- **描述：**用户输入“home”，系统触发“回家模式”，打开所有灯光、根据当前温度调节空调（如温度 $> 28^{\circ}\text{C}$ ，设为 22°C ），提示“Home mode activated”。
- **参与者：**用户
- **交互流程：**
 1. 用户输入“home”。
 2. 系统触发“回家模式”场景。
 3. 系统打开所有灯光，调节空调温度。
 4. 系统提示“Home mode activated”。
- **对象：**用户、控制中心、场景、灯光、空调、传感器。

需求分析：用例分析

- 用例3：离家模式

- **描述：**用户输入“away”，系统触发“离家模式”，关闭所有设备，提示“Away mode activated”。
- **参与者：**用户
- **交互流程：**
 1. 用户输入“away”。|
 2. 系统触发“离家模式”场景。
 3. 系统关闭所有灯光和空调，提示“Away mode activated”。
- **对象：**用户、控制中心、场景、灯光、空调。

需求分析：用例分析

- 用例4：温度自动调节
 - **描述：**系统周期性检查温度，若 $> 28^{\circ}\text{C}$ ，空调设为 22°C 并开启；
若 $< 20^{\circ}\text{C}$ ，设为 26°C 并开启；否则关闭空调。
 - **参与者：**无（自动化）
 - **交互流程：**
 1. 系统获取温度传感器数据。
 2. 系统根据温度调节空调。
 3. 系统更新空调状态。
 - **对象：**传感器、控制中心、空调。

需求分析：用例分析

- 用例5：定时开关灯
 - **描述：**系统每小时自动切换灯光状态（开→关或关→开）。
 - **参与者：**无（自动化）
 - **交互流程：**
 1. 系统检查定时器。
 2. 若到达时间阈值，切换灯光状态。
 - **对象：**定时器、控制中心、灯光。

2.识别类和对象

- **目标：** 从需求和用例中找出系统中涉及的对象，并抽象为类。

基于用例分析，识别以下核心对象和对应的类：

- **用户 (User)**：发送指令（如“home”），表示用户交互。
- **控制中心 (SmartHomeHub)**：接收指令、协调设备和场景，核心管理对象。
- **灯光 (Light)**：可开关的设备，执行“light_on/off”操作。
- **空调 (AirConditioner)**：可调节温度的设备，支持自动调节。
- **传感器 (Sensor)**：提供温度数据，支持“温度自动调节”。
- **场景 (Scene)**：管理“回家模式”和“离家模式”，批量操作设备。
- **定时器 (Timer)**：管理定时任务，触发灯光切换。
- **设备 (Device)**：灯光和空调的抽象基类，统一设备接口。

OOP意义:

- 用例中的对象直接映射为类，如“灯光”对应 Light 类，“回家模式”对应 Scene 类。
- 抽象基类 Device 从“灯光”和“空调”的共性中提取，体现OOP的抽象化。
- “控制中心”作为协调者，提示组合关系。

3. 定义类的属性和行为

- **目标：** 确定每个类的属性（数据）和行为（方法），确保职责单一。

- Device (抽象基类):

- 属性:

- -name: str (设备名称, 如“Living Room Light”)
 - -status: bool (设备状态, True/False)

- 行为:

- +turn_on(): void (抽象, 打开设备)
 - +turn_off(): void (抽象, 关闭设备)
 - +get_status(): bool (获取状态)

- Light:
 - 继承: Device
 - 属性: 无额外属性
 - 行为:
 - +turn_on(): void (设置 _status=True, 打印状态)
 - +turn_off(): void (设置 _status=False, 打印状态)
 - +get_status(): bool (返回 _status)

- AirConditioner:

- 继承: Device

- 属性:

- -temperature: int (当前温度)

- 行为:

- +turn_on(): void (设置 _status=True, 打印状态)

- +turn_off(): void (设置 _status=False, 打印状态)

- +set_temperature(temp: int): void (设置温度)

- +auto_adjust(current_temp: float): void (根据温度调节)

- Sensor:
 - 属性:
 - -type: str (传感器类型, 如“temperature”)
 - -value: float (温度数据)
 - 行为:
 - +get_value(): float (获取温度)

- Timer:
 - 属性:
 - -start_time: float (计时起点)
 - 行为:
 - +check_timer(device: Device, interval: float): void (检查时间, 触发设备操作)

- Scene:
 - 属性:
 - -name: str (场景名称, 如"home")
 - -devices: list[Device] (设备列表)
 - 行为:
 - +apply_mode(mode: str): void (应用场景, 如"home")

- SmartHomeHub:

- 属性:

- -devices: list[Device] (设备列表)
 - -scene: Scene (场景管理)
 - -timer: Timer (定时器)
 - -sensor: Sensor (传感器)

- 行为:

- +add_device(device: Device): void (添加设备)
 - +process_command(command: str): void (处理指令)
 - +receive_temperature(temp: float): void (处理温度数据)
 - +check_timer(): void (检查定时任务)
 - +check_temperature(): void (触发温度检查)

- User:

- 属性:

- -username: str (用户名)

- 行为:

- +send_command(hub: SmartHomeHub, command: str): void (发送指令)

4. 确定类之间的关系

- **目标：**构建类的结构层级及其相互关系，包括继承、组合和关联。

- 继承：

- Light 和 AirConditioner 继承 Device，实现多态（如统一调用 turn_on）。
- **依据：**用例中的设备（如灯光、空调）有共性（如开关操作），抽象为 Device 基类。
- **示例：**Scene.apply_mode("home") 通过 Device 接口操作不同设备。

- 组合：

- SmartHomeHub

包含：

- `_devices: list[Device]`（管理所有设备）
 - `_scene: Scene`（管理场景）
 - `_timer: Timer`（管理定时任务）
 - `_sensor: Sensor`（管理传感器数据）

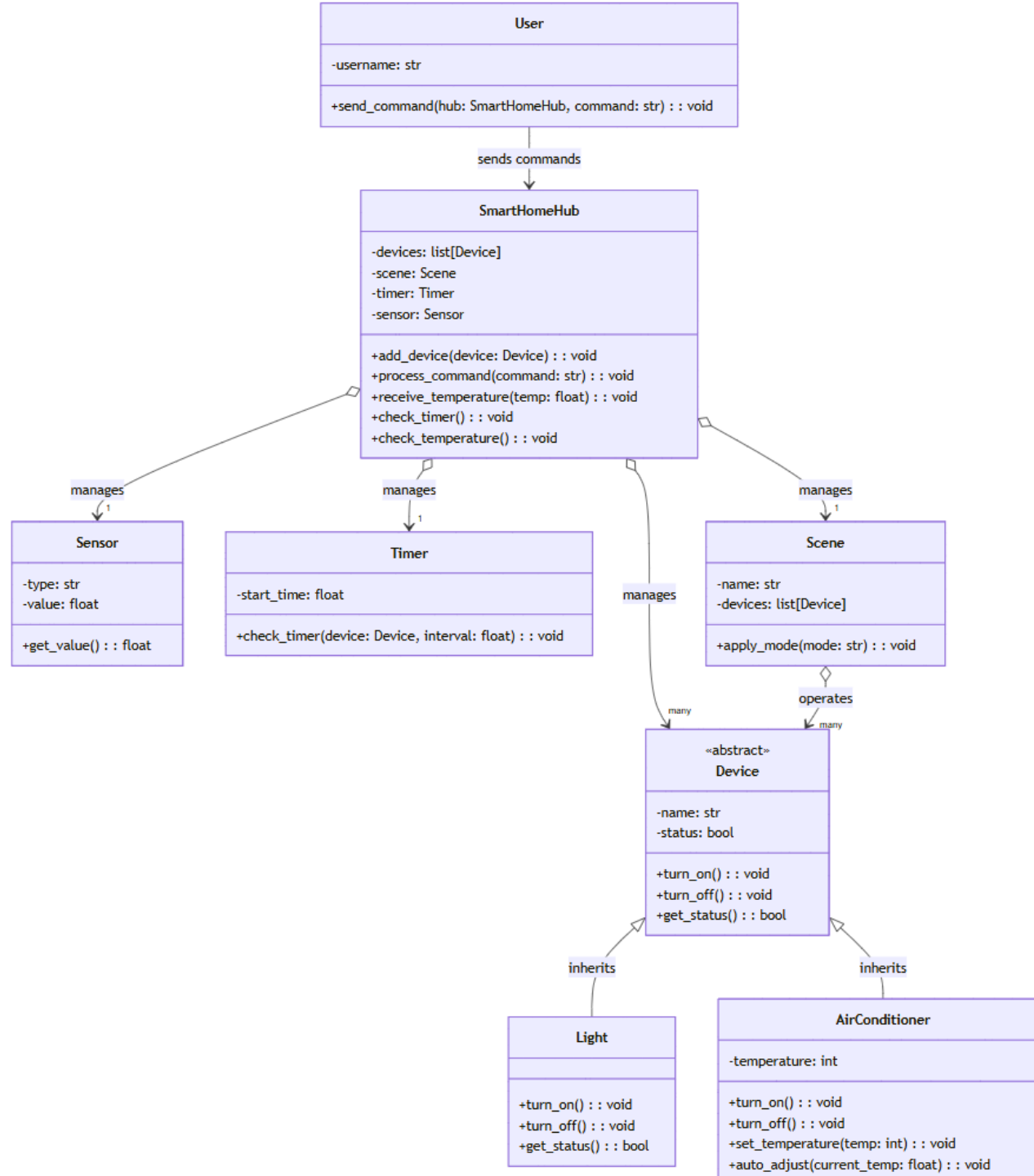
- Scene 包含 `_devices: list[Device]`（场景操作的设备列表）。

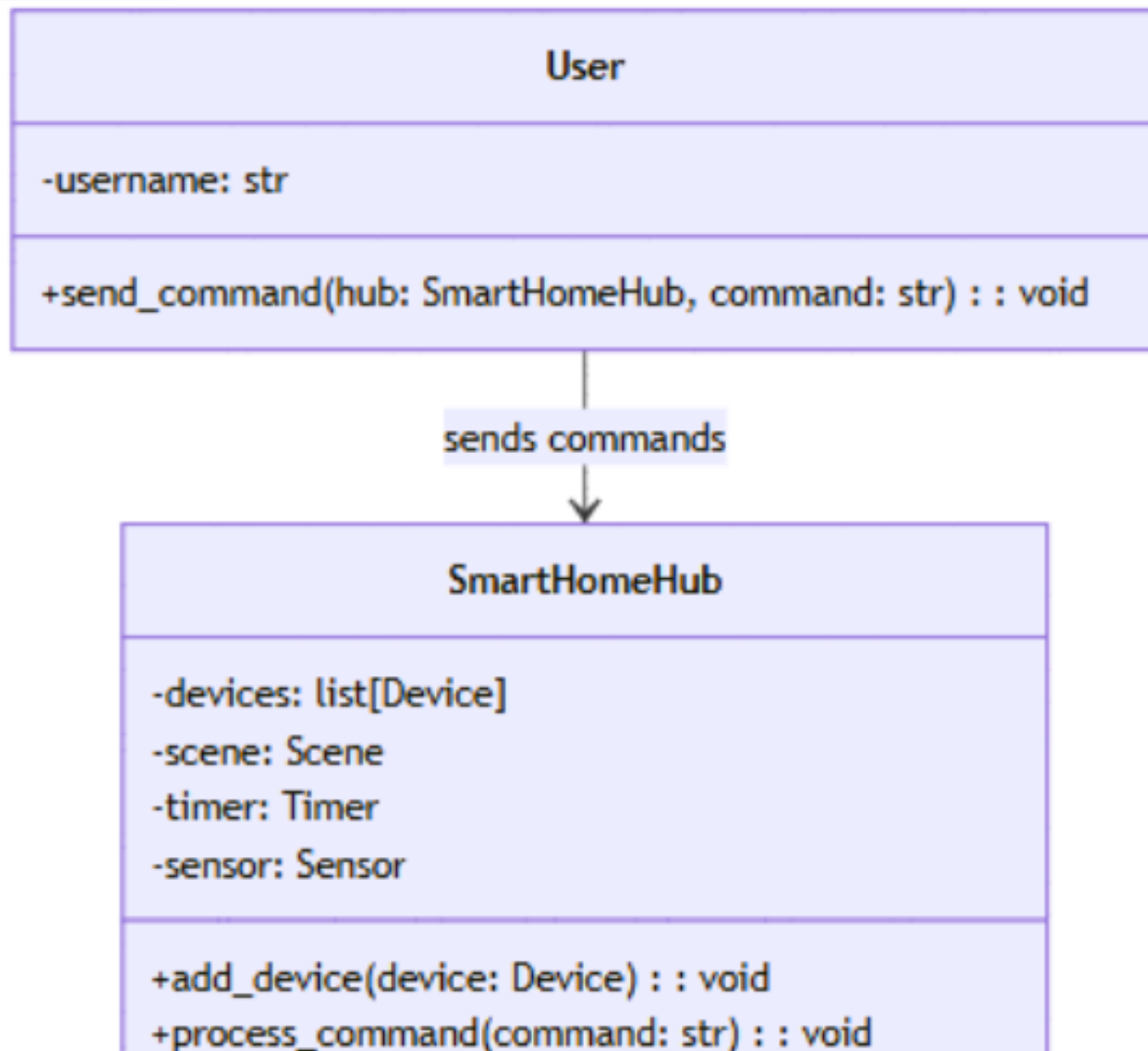
- **依据：**用例中 SmartHomeHub 协调多个对象，“回家模式”涉及批量设备操作。

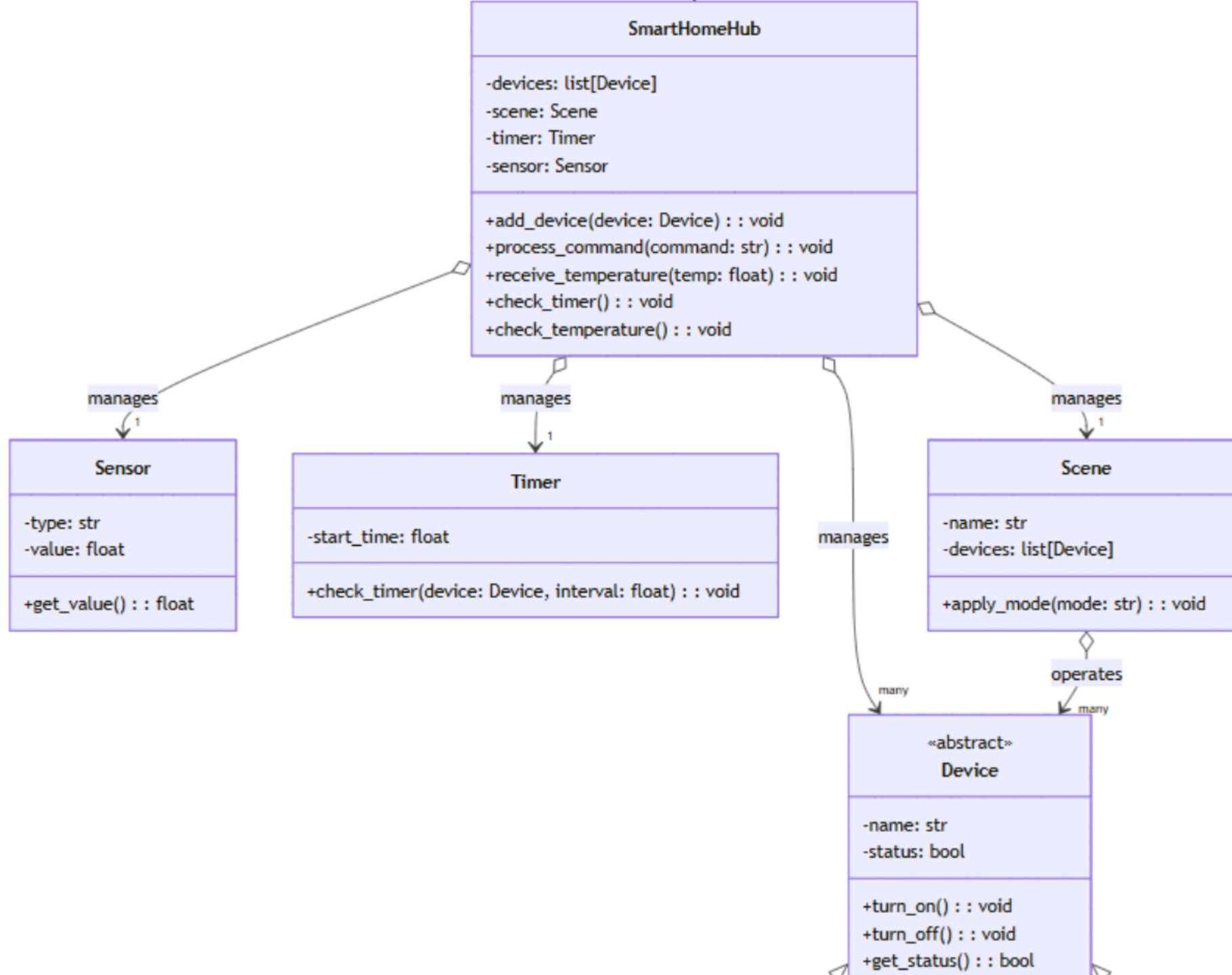
- **示例：**SmartHomeHub 调用 Scene.apply_mode 实现“回家模式”。

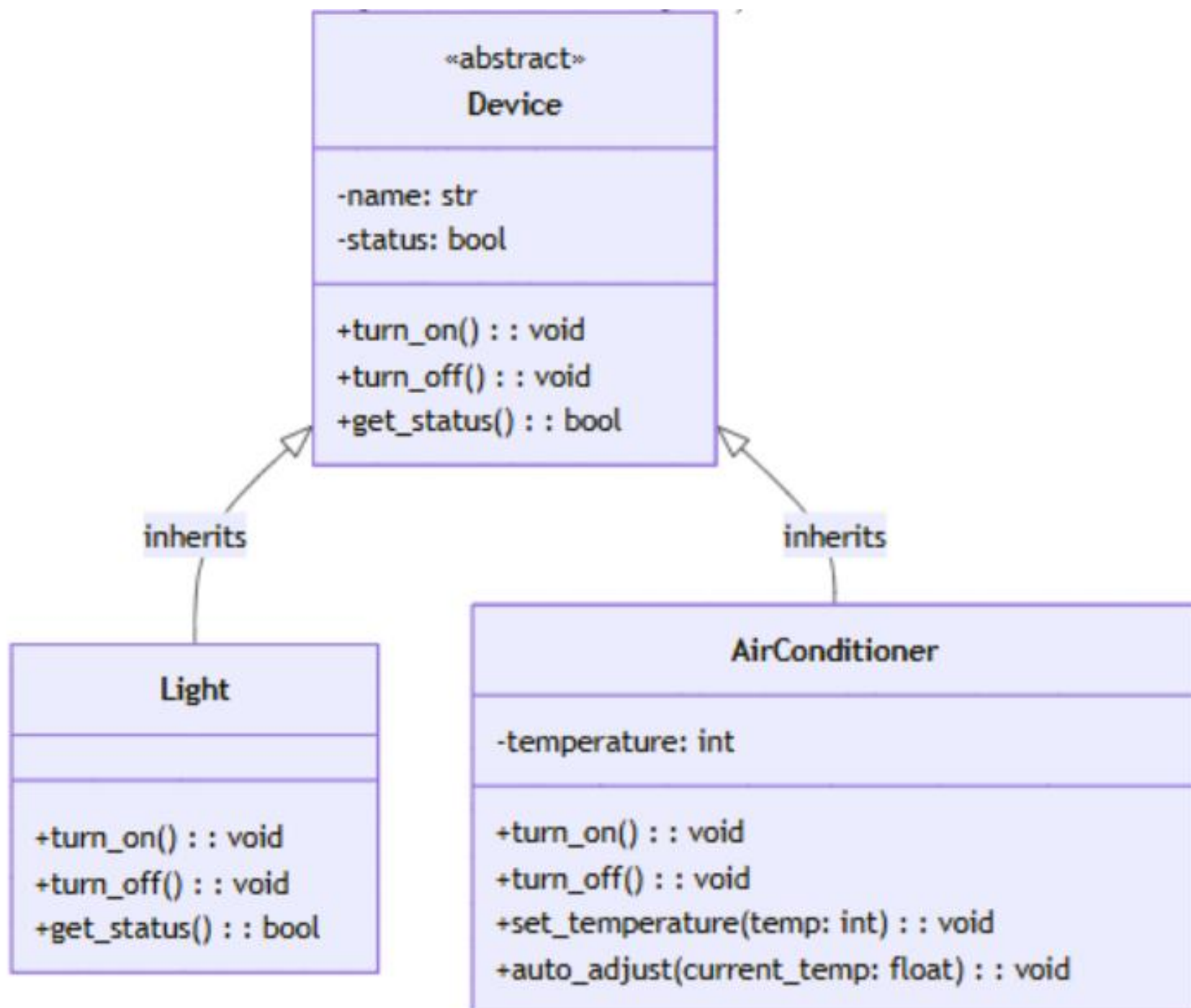
- 关联：
 - User 通过 send_command 与 SmartHomeHub 交互。
 - **依据：**用例中用户通过指令（如“home”）触发系统操作。
 - **示例：**User.send_command(hub, "home") 发起场景。

- **类图：**类图设计阶段主要关注类的静态结构和关系。









OOP意义:

- 继承支持多态，统一设备操作接口。

```
for 每个设备e:  
    e.tuon_on()
```

- 组合实现模块化管理，SmartHomeHub 协调子模块。

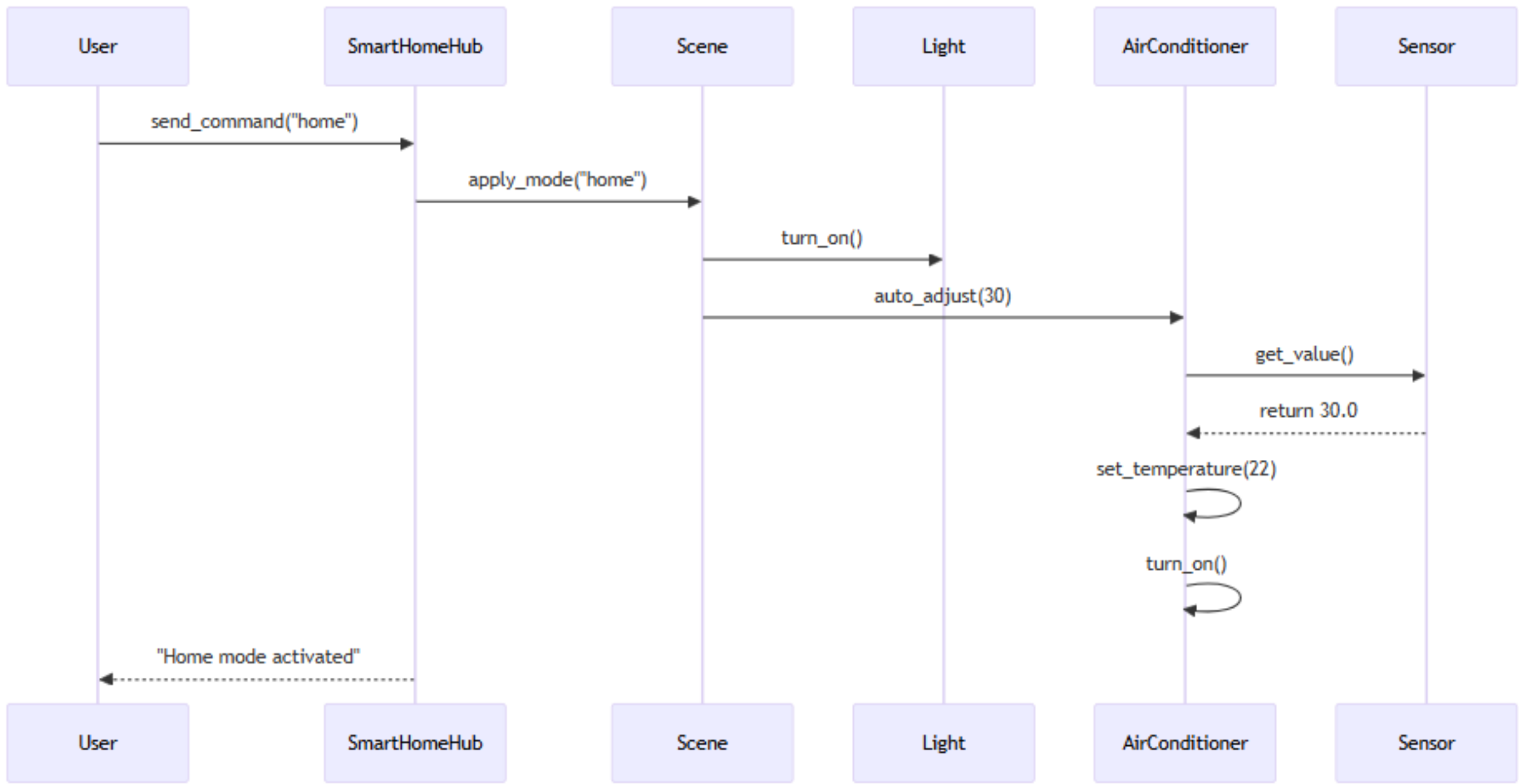
```
class SmartHomeHub{  
    vecotr<Device> devices;  
    vecotr<Scene> scenes;  
    Sensor sensor;    // vecotr<Sensor> sensors;  
    Timer timer;      //vecotr<Timer> timers;  
    //...  
};
```

- 关联明确交互路径，如用户与系统的指令传递。

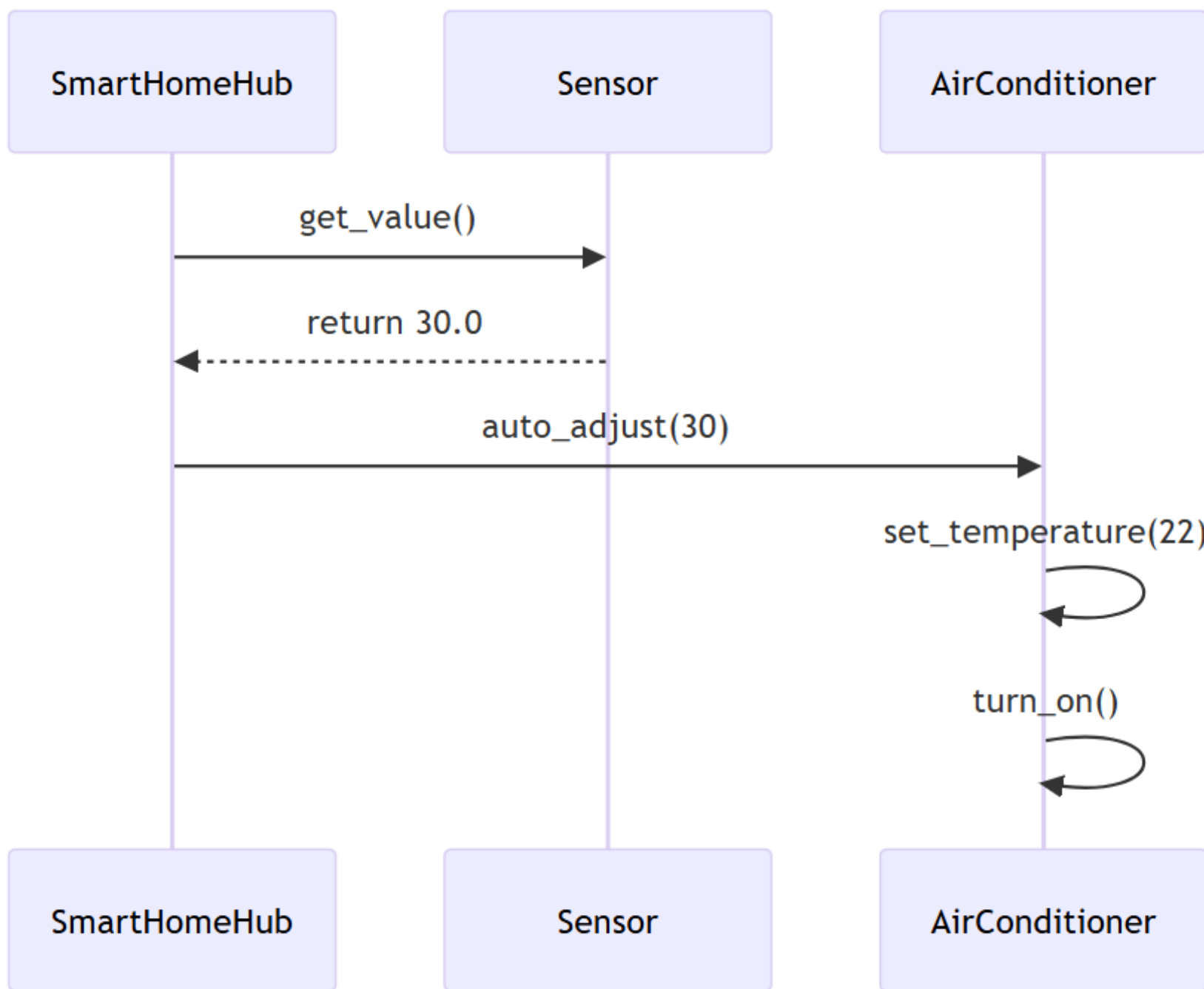
```
User send_command to SmartHomeHub  
Scene turn_on Light
```

5. 设计对象的交互

- **目标：**明确对象如何协作完成系统任务，使用序列图描述动态交互。



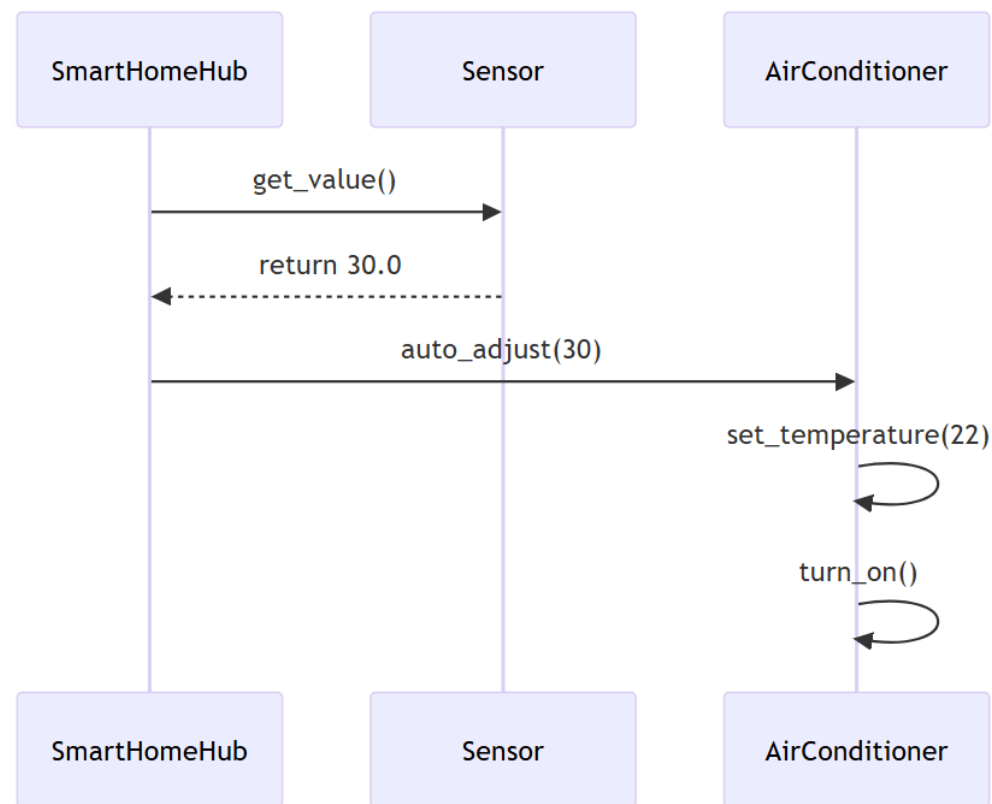
展示了“回家模式”用例中，对象之间如何协作完成任务



展示了“温度自动调节”用例中，对象之间如何协作完成任务

OOP意义:

- 对象交互通过方法调用实现，数据流封装在对象中。
- 多态性（如 Device.turn_on）简化协作逻辑。
- 相比过程式编程，OOP交互模块化，职责清晰。



6. 系统架构与模块划分

- **目标：**组织类与对象到模块或子系统中，体现OOP 的模块化设计。

模块划分：

- 设备管理模块：
 - **功能：**管理灯光、空调等设备。
 - **类：**Device、Light、AirConditioner。
 - **示例：**AirConditioner.auto_adjust 实现“温度自动调节”。

- 场景管理模块：
 - **功能：** 管理“回家模式”和“离家模式”。
 - **类：** Scene.
 - **示例：** Scene.apply_mode("home") 批量操作设备。
- 指令处理模块：
 - **功能：** 解析用户指令。
 - **类：** SmartHomeHub.
 - **示例：** SmartHomeHub.process_command("home").

- 传感器数据采集模块：
 - **功能：** 获取温度数据。
 - **类：** Sensor.
 - **示例：** Sensor.get_value 支持“温度自动调节”。
- 自动化控制模块：
 - **功能：** 根据传感器数据调节设备。
 - **类：** SmartHomeHub、AirConditioner.
 - **示例：** SmartHomeHub.receive_temperature.

- 定时任务模块：
 - **功能：**管理定时开关灯。
 - **类：**Timer.
 - **示例：**Timer.check_timer.
- 用户交互模块：
 - **功能：**处理用户指令。
 - **类：**User.
 - **示例：**User.send_command.

7. 重构与优化设计

8. 编码实现（示例）

```
// 抽象基类 Device
class Device {
public:
    virtual void turn_on() = 0;
    virtual void turn_off() = 0;
    virtual ~Device() = default; // 确保多态析构
};
```

```
// Light 类
class Light : public Device {
private:
    std::string _name;
    bool _status{false};

public:
    Light(const std::string& name) : _name{name} {}

    void turn_on() override {
        _status = true;
        std::cout << _name << " is ON\n";
    }

    void turn_off() override {
        _status = false;
        std::cout << _name << " is OFF\n";
    }

    bool get_status() const { return _status; }
};
```

```
// SmartHomeHub 类
class SmartHomeHub {
private:
    std::vector<std::unique_ptr<Device>> _devices;
    Timer _timer;
    std::unique_ptr<Scene> _scene;
    Sensor _sensor;

public:
    SmartHomeHub() : _scene{nullptr} {}

    void add_device(std::unique_ptr<Device> device) {
        _devices.push_back(std::move(device));
        if (!_scene) {
            _scene = std::make_unique<Scene>(_devices);
        }
    }
}
```

// 示例运行

```
int main() {  
    SmartHomeHub hub;  
    hub.add_device(std::make_unique<Light>("Living Room Light"));  
    hub.add_device(std::make_unique<AirConditioner>("Bedroom AC"));  
    User user("Alice");  
    user.send_command(hub, "home"); // 触发回家模式  
    hub.check_temperature(); // 触发温度自动调节  
    return 0;  
}
```

8.2.2 过程式编程和面向对象编程的优缺点

过程式编程的优缺点

优点：

1. 实现简单：

- **特点：**以功能为中心，函数分解任务，逻辑线性。
- **示例：**智能家居系统中，“回家模式”通过 `activateScene("home")` 操作全局 `devices` 数组，快速实现。
- **适用场景：**适合小型系统、原型开发或教学场景，初学者易于掌握算法逻辑。

2. 代码直观：

- **特点：**函数直接反映功能步骤，流程清晰。
- **示例：**`controlDevice("light_on")` 修改灯光状态，逻辑直白。
- **适用场景：**快速编码或单线程、无状态管理的简单任务。

3. 执行效率高：

- **特点：**避免对象创建和方法调用开销，资源占用少。
- **示例：**adjustAC(temp) 直接访问全局数据，执行路径简单。
- **适用场景：**性能敏感的嵌入式系统或实时控制任务。

缺点：

1. 扩展性差：

- ****特点****：功能与数据耦合，新增功能需修改全局逻辑。
- **示例**：添加窗帘需修改 devices 和 controlDevice，易引入错误。
- **问题**：违反开闭原则，系统规模增大时难以扩展。

2. 耦合高：

- **特点**：函数共享全局数据，状态管理复杂。
- **示例**：activateScene("home") 和 adjustAC 修改同一 devices 数组，可能导致并发冲突。
- **问题**：调试困难，尤其在多线程环境中。

3. 维护性低:

- **特点:** 逻辑分散, 复杂系统难以维护。
- **示例:** 新增湿度调节需重构 `adjustAC`, 增加测试成本。
- **问题:** 代码规模增大后, 修改成本高。

4. 重用性差:

- **特点:** 函数逻辑分散, 难以复用。
- **示例:** `controlDevice` 的灯光逻辑无法直接复用。
- **问题:** 需重复开发, 尽管函数库可部分缓解。

面向对象编程（OOP）的优缺点

优点：

1. 封装性强：

- **特点：**数据和行为封装，私有属性通过方法访问。
- **示例：** `Light.status` 和 `AirConditioner.temperature` 私有，`turn_on` 和 `auto_adjust` 控制访问。
- **优势：** 提高安全性，支持异常处理，降低错误。

2. 扩展性好：

- **特点：** 继承和多态支持新功能，符合开闭原则。
- **示例：** 新增 `Curtain` 继承 `Device`，自动纳入“回家模式”。
- **优势：** 适合复杂、长期维护的项目。

3. 维护性高：

- **特点：**模块化设计，类职责单一。
- **示例：**修改 `AirConditioner.auto_adjust` 不影响其他类。
- **优势：**调试成本低，适合中大型系统。

4. 重用性强：

- **特点：**类和方法可复用，支持框架和设计模式。
- **示例：**`Device.turn_on` 被复用，“回家模式”通过 `Scene` 复用逻辑。
- **优势：**减少重复开发。

5. 模块化设计：

- **特点：**系统分解为对象，耦合低。
- **示例：**`Sensor`、`SmartHomeHub`、`AirConditioner` 协作实现“温度自动调节”。
- **优势：**清晰的层次结构，易于团队协作。

缺点：

1. 初期设计复杂：

- **特点：**需定义类、关系和方法，耗时。
- **示例：**实现“回家模式”需设计 Scene、SmartHomeHub、Device。
- **问题：**小型项目可能过度设计。

2. 运行开销较高：

- **特点：**对象实例化、虚函数调用增加开销。
- **示例：**SmartHomeHub 管理多个对象，资源占用较多。
- **问题：**资源受限的嵌入式系统可能不适用，但现代硬件影响较小。

3. 学习曲线陡峭：

- **特点：**需理解封装、继承、多态及SOLID原则。
- **示例：**将“温度自动调节”映射为类交互较抽象。
- **问题：**对新手不友好。

特性	过程式编程	面向对象编程
设计方式	以功能为中心，分解为函数，数据集中管理。	以对象为中心，分解为类，数据和行为封装。
代码组织	函数分散在文件中，缺乏层次结构。	类和包形成清晰层次，易于管理。
示例实现	“回家模式”：activateScene("home") 操作全局 devices。	“回家模式”：Scene.apply_mode("home") 调用多态方法。
扩展性	差，需修改全局逻辑。	好，通过继承和组合扩展。
耦合性	高，共享全局数据。	低，方法调用交互。
维护性	低，逻辑分散。	高，职责单一。
适用场景	小型系统、快速原型、嵌入式性能敏感任务。	中大型系统、需要扩展和维护的项目。
性能开销	低，函数调用高效。	较高，对象创建和方法调用有开销。

8.3 类与对象

- 8.3.1 类的定义
- 8.3.2 定义类的对象（变量）
- 8.3.3 成员访问运算符 .
- 8.3.4 间接访问运算符 ->
- 8.3.5 struct和class区别
- 8.3.6 添加成员函数：让对象拥有行为
- 8.3.7 this 指针
- 8.3.8 类对象的大小

8.3.1 类的定义

- 是用于描述一组具有共同特征和行为的对象的**模板**或**蓝图**。这些共同特征包括**数据属性**（如姓名、颜色、速度等）和**行为**（如走路、加速、打印等）。
- 类比作“汽车的设计图纸”，而对象则是根据设计图制造出的具体汽车。
- string 类描述了所有字符串对象共有的属性和行为，而一个具体的 string 类

```
#include <string>
using std::string;
int main(){
    string str("hello world!");
}
```

类的基本组成

- 属性 (**Attributes**) : 表示对象的状态或数据, 例如一个人的姓名、年龄。
- 行为 (**Behaviors**) : 表示对象能够执行的操作, 例如人可以走路、说话。

class 或 struct

- 类是使用关键字 `class` 或 `struct` 定义的一种用户自定义数据类型

```
struct student {  
    string name;  
    double score;  
};
```

`student` 类中的 `name` 和 `score` 也被称为**数据成员**（或**成员变量**）。

定义了一个表示日期的类 Date，它包含了 year、month 和 day 三个数据成员，并为它们提供了默认值：

```
struct Date {  
    int year{2000};    // 年份，默认值为 2000  
    int month{1};      // 月份，默认值为 1  
    int day{1};        // 日期，默认值为 1  
};
```

8.3.2 定义类的对象（变量）

- 与内置类型类似，程序员可以使用类来定义变量。这些变量在 C++ 中也称为**对象**，是类的具体实例，因此也叫做**类的实例**。

下面的代码定义了 `Date` 类的两个对象 `d1` 和 `d2`：

```
Date d1;                // year = 2000, month = 1, day = 1
Date d2{2024, 3, 6};    // year = 2024, month = 3, day = 6
```

每个对象都拥有自己独立的数据成员：在对象 `d1` 中，`year`、`month` 和 `day` 分别采用默认值；而在对象 `d2` 中，这些数据成员由花括号 `{2024, 3, 6}` 显式初始化。

8.3.3 成员访问运算符 .

- 通过成员访问运算符 . 来访问或修改该对象的成员变量

```
#include <iostream>
struct Date {
    int year{ 2000 }, month{ 1 }, day{ 1 };
};
int main() {
    Date day;           // day 是 Date 类的对象
    day.year = 2018;    // 通过成员访问运算符 . 修改成员 year
    day.month = 6;
    day.day = 1;
    std::cout << day.year << '-' << day.month << '-' << day.day << '\n';
}
```

8.3.4 间接访问运算符->

- 当我们通过指针变量来访问对象的成员时，不能使用点号 .，而需要使用间接访问运算符 ->。
- 通过指向 Date 对象的指针 date 来访问其成员：

```
void print(Date* date) {  
    // date 是 Date* 类型的指针，不是对象本身  
    // 使用 -> 运算符访问 date 指向的对象的成员  
    std::cout << date->year << "-" << date->month << "-" << date->day << '\n';  
}
```



```
#include <iostream>

struct Date {
    int year{ 2000 }, month{ 1 }, day{ 1 };
};

void print(Date *date) { // date是Date*指针类型，不是一个Date对象
    // 间接访问运算符->访问指针变量date指向的Date对象的成员year, month和day
    std::cout << date->year << "-" << date->month << "-" << date->day << '\n';
}

int main() {
    Date day;
    day.year = 2018; // 通过成员访问运算符.访问类对象day的成员year
    day.month = 6;
    day.day = 1;
    print(&day); // 将day的地址作为指针传给print函数
}
```

8.3.5 struct和class区别

- 也可以使用 class 关键字来定义类

```
#include <iostream>
class Date {
    int year{ 2000 }, month{ 1 }, day{ 1 }; // 默认是 private
};
int main() {
    Date day;
    day.year = 2018; // 编译错误, 无法访问 private 成员
    day.month = 6;
    day.day = 1;
}
```

```
#include <iostream>
class Date {
    int year{ 2000 }, month{ 1 }, day{ 1 }; // 默认是 private
};
int main() {
    Date day;
    day.year = 2018; // 编译错误, 无法访问 private 成员
    day.month = 6;
    day.day = 1;
}
```

编译器会报错:

```
error C2248: "Date::year": 无法访问 private 成员
```

这是因为：

- 使用 `class` 定义的类中，成员变量默认是 **private**（私有的），
- 私有成员只能在类的成员函数内部访问，外部函数（如 `main()`）无法访问。

而如果使用 `struct` 定义类，如前面的示例那样，类成员默认是 **public**（公开的），因此可以在外部自由访问对象的成员。

- 要想让 class 中的成员也能被外部访问，可以使用访问控制修饰符 public 明确声明这些成员为公开的：

```
#include <iostream>

class Date {
public: // 下面的成员都是 public 的
    int year{ 2000 }, month{ 1 }, day{ 1 };
};

void print(Date* date) {
    std::cout << date->year << "-" << date->month << "-" << date->day << '\n';
}

int main() {
    Date day;
    day.year = 2018;
    day.month = 6;
    day.day = 1;
    print(&day);
}
```

- 虽然 struct 和 class 在语法上几乎完全一致，但它们的默认访问权限不同。
- 在现代 C++ 开发中，推荐使用 class 定义类，并明确使用 public、private 和 protected 修饰符来控制成员访问权限。
- struct 更常用于表示只包含数据的简单结构体，用于与 C 语言代码兼容或表示数据聚合。

8.3.6 添加成员函数：让对象拥有行为

- 在面向对象编程中，对象不仅拥有属性（数据），还应具备行为（操作能力）。为了让对象能够执行操作，我们需要在类中添加成员函数（member function），它定义了对象可以做什么。
- 在 Date 类中添加了一个名为 print 的成员函数，用于显示日期信息：

```
class Date {  
public:  
    int year{ 2000 }, month{ 1 }, day{ 1 };  
  
    void print() { // 显示日期信息  
        std::cout << year << "-" << month << "-" << day << std::endl;  
    }  
};
```

- 当然，也可以通过指向对象的指针来调用成员函数。这时需要使用间接访问运算符 ->：

```
void print(Date* date) { // date 是 Date 类型的指针  
    date->print();        // 通过指针调用成员函数  
}
```


- 可以像访问成员变量一样，通过成员访问运算符 . 调用成员函数：

```
int main() {  
    Date day;  
    day.year = 2018;  
    day.month = 6;  
    day.day = 1;  
  
    day.print(); // 调用 print() 函数，显示日期信息  
  
    return 0;  
}
```

8.3.7 this 指针

- 编译器在处理成员函数时，会将它们转换为普通函数，并自动添加一个额外的参数 `this`，用来指向调用该函数的对象。

```
void print();
```

在编译后可能会被转换为：

```
void print(Date* this);
```

- 调用 `day.print()` 实际上是将对象 `day` 的地址作为 `this` 参数传入函数内部。因此，在成员函数中访问成员变量 `year`、`month`、`day`，其实等价于 `this->year`、`this->month`、`this->day`。
- 对于 `Date` 类，编译器会将 `print()` 成员函数转换为如下的普通函数：

```
void print(Date* this) {  
    std::cout << this->year << '-' << this->month << '-' << this->day << '\n';  
}
```

这就解释了为什么非静态成员函数必须通过对象来调用 —— 因为 `this` 指针指向该对象。

- 因此，在类的非静态成员函数中可以使用this指针（但函数的参数里不能声明this指针）。

```
class Date {  
public:  
    int year{ 2000 }, month{ 1 }, day{ 1 };  
    void print()    //注意: 不能写成: void print(Date *this)  
    {  
        //通过指针变量this访问其指向对象的year、month、day成员  
        std::cout << this->year << "-" << this->month << "-" << this->day << '\n';  
    }  
};
```

返回 this：支持链式调用

- 一个类的非静态成员函数可以返回 this 指针或 this 指针指向的对象的引用。
- 这样做的主要目的是支持方法链调用（Method Chaining），使多个成员函数的调用可以串联在一起。

```
int main() {  
    Date day;  
    day.setYear(2019).setDay(20); // 支持链式调用  
    day.print();  
}
```

```
class Date {
public:
    int year{ 2000 }, month{ 1 }, day{ 1 };
    void print() { std::cout << year << "-" << month << "-" << day << '\n'; }

    Date& setDay(int d) {
        day = d;
        return *this; // 返回调用对象的引用
    }

    Date& setYear(int y) {
        year = y;
        return *this; // 返回调用对象的引用
    }
};

int main() {
    Date day;
    day.setYear(2019).setDay(20); // 支持链式调用
    day.print();
}
```

```
class Date {  
public:  
    int year{ 2000 }, month{ 1 }, day{ 1 };  
    void print() { std::cout << year << "-" << month << "-" << day << '\n'; }  
  
    Date& setDay(int d) {  
        day = d;  
        return *this; // 返回调用对象的引用  
    }  
  
    Date* setYear(int y) {  
        year = y;  
        return this; // 返回调用对象的指针  
    }  
};  
  
int main() {  
    Date day;  
    day.setYear(2019)->setDay(20); // setYear() 返回指针, 必须用 -> 调用 setDay()  
    day.print();  
}
```

8.3.8 类对象的大小

- 一个类对象占据的内存主要用于存放其数据成员。因此，类对象的大小通常等于或略大于所有数据成员占据内存之和。
- 为什么类对象的大小略大于数据成员的内存之和呢？

这是因为数据在内存中是按照一定规则进行对齐的，以提高 CPU 访问数据的效率。

8.3.8 类对象的大小

对齐规则通常是：

- 每个数据成员的起始地址通常是其类型大小的整数倍（即对齐边界），以便于 CPU 更高效地访问。（例如 int 通常需要 4 字节对齐，double 通常需要 8 字节对齐）。
- 整个类对象的大小必须是最大数据成员对齐要求的整数倍，以确保每个成员都可以正确对齐。

```
class X {  
    int a;    // 4 字节  
    short b;  // 2 字节  
    int c;    // 4 字节  
    double d; // 8 字节  
};
```

1. `int a` 占用 4 字节，且 `int` 通常需要 4 字节对齐，假设对象从地址 0 开始分配内存，则 `a` 占据地址 0~3。
2. `short b` 占用 2 字节，`b` 放在地址 4~5（占两个字节）；然后为了让 `c` 从地址 8 开始，对其后进行 2 字节（地址 6~7）填充。
3. `int c` 占用 4 字节，放在地址 8~11，满足其 4 字节对齐要求。
4. `double d` 占用 8 字节，且 `double` 需要 8 字节对齐。因此，编译器会在 `c` 后面插入 4 字节的填充字节，以确保 `d` 从地址 16 开始。



成员函数与类对象内存

- 类的成员函数（非静态成员函数）通常在内存中只有一份拷贝，这些函数由类的所有对象共享。当通过对象调用成员函数时，编译器会隐式地将对象的地址（即 `this` 指针）传递给成员函数，使得成员函数能够访问和操作该对象的数据成员。
- 因此，每个对象的数据成员占据独立的内存，而成员函数的代码是所有对象共享的。即使一个类中定义了多个成员函数，所有对象仍共享同一份函数代码，这些函数不会增加类对象的大小。

sizeof 运算符

- sizeof() 运算符的返回值是类型或对象占据的内存大小，单位为字节（byte）。
- 与内置类型一样，可以使用sizeof() 运算符来检查一个类对象占据内存的大小

```
#include <iostream>
class X {
    int a, b, c;
    double d;
};
int main() {
    X x;
    std::cout << sizeof(int) << '\t' << sizeof(double) << '\n';
    std::cout << sizeof(x) << '\t' << sizeof(X) << '\n';
}
```

```
#include <iostream>
class X {
    int a, b, c;
    double d;
};
int main() {
    X x;
    std::cout << sizeof(int) << '\t' << sizeof(double) << '\n';
    std::cout << sizeof(x) << '\t' << sizeof(X) << '\n';
}
```

程序运行结果：

4	8
24	24

假如程序中定义了两个 `Date` 类的对象 `day` 和 `day2`：

```
int main(){  
    Date day, day2;  
    day.year = 2018;  
    day.month = 6;  
    day.day = 1;  
}
```

day	
year	2008
month	6
day	1

day2	
year	2000
month	1
day	1

Date

```
print(){  
    ...  
}
```

```
setDay(){  
    ...  
}
```

```
setYeate(){  
    ...  
}
```

```
...
```

8.4 构造函数

- 8.4.1 什么是构造函数?
- 8.4.2 初始化成员列表
- 8.4.3 初始化成员列表 vs 类内成员初始化器
- 8.4.4 拷贝构造函数
- 8.4.5 赋值运算符: `operator=`
- 8.4.6 深拷贝与浅拷贝
- 8.4.7 `delete`
- 8.4.8 隐式类型转换、`explicit`
- 8.4.9 类对象数组

8.4 构造函数

- 8.4.10 委托构造函数
- 8.4.11 类体外定义成员函数和构造函数
- 8.4.12 头文件与源文件的分离

8.4.1 什么是构造函数？

- 类是用来设计对象的蓝图，而**构造函数**则负责在创建对象时赋予它们初始的属性和状态。

```
Date day;
```

- 当定义 Date 类的对象 day 时，系统会分配内存来存放 day，然后自动调用 Date 类的构造函数，对 day 的属性进行初始化。

类内成员初始化 (in-class member initialization)

- 在定义类的数据成员（例如 Date 类的 year、month、day）时，可以使用成员初始化器直接赋予它们默认值，被称为 成员初始化器 (member initializer) 或者 类内成员初始化 (in-class member initialization)。

```
class Date {  
private: // 默认就是私有的，可以省略private  
    int year{2000};  
    int month{1};  
    int day=1;
```

没有显式初始化的情况下，所有类对象的类内初始化数据成员都将具有相同的默认值。

```
#include <iostream>

class Date {
private: // 默认就是私有的，可以省略private
    int year{2000};
    int month{1};
    int day=1;

public:
    void print() const { // const成员函数，表明不修改对象状态
        std::cout << year << "-" << month << "-" << day << '\n';
    }
};

int main() {
    Date day, day1;
    day.print();
    day1.print();
}
```

运行结果：

2000-1-1

2000-1-1

构造函数

- 实际上，在定义类对象时，编译器会自动调用一个特殊的成员函数，称为**构造函数**，来对类对象的数据成员进行初始化。
- **构造函数**是一种函数名*与类名相同且没有返回类型*的成员函数。
- 如果程序员没有显式定义构造函数，编译器会自动生成一个**默认构造函数**，其参数列表为空，函数体也为空。

默认构造函数

- 这种不带参数（或所有参数都具有默认值）的构造函数叫做**默认构造函数**。

```
class Date {  
private:  
    int year{2000};  
    int month{1};  
    int day{1};  
  
public:  
    Date() {} // 构造函数名与类名相同，没有返回类型  
              // 默认构造函数没有任何参数和函数体代码  
    void print() const {  
        std::cout << year << "-" << month << "-" << day << '\n';  
    }  
};
```

- 为了验证定义类对象是否会自动调用构造函数，可以在构造函数里添加一些输出语句：

```
class Date {  
    private:  
        int year{2000};  
        int month{1};  
        int day{1};  
  
    public:  
        Date() {  
            std::cout << "构造 Date 对象: " << std::endl;  
            print();  
        }  
        void print() const {  
            std::cout << year << "-" << month << "-" << day << '\n';  
        }  
};  
  
int main() {  
    Date day; // 定义类对象时自动调用匹配的构造函数  
}
```

- 定义 Date 对象 day 时会自动调用默认构造函数 Date()。执行该程序时，屏幕上将显示：

```
构造 Date 对象：
```

```
2000-1-1
```

即先执行第一句输出，然后调用 `print()` 成员函数。

```
Date() {  
    std::cout << "构造 Date 对象：" << std::endl;  
    print();  
}
```

- 可以定义带输入参数的构造函数，在定义类对象时，使用输入的实际参对类对象的数据成员进行初始化：

```
class Date {  
private:  
    int year{2000};  
    int month{1};  
    int day{1};  
public:  
    Date(int y, int m, int d) {  
        year = y;  
        month = m;  
        day = d;  
        std::cout << "构造 Date 对象: \t";  
        print();  
    }  
    void print() const {  
        std::cout << year << "-" << month << "-" << day << '\n';  
    }  
};
```



```
class Date {
private:
    int year{2000};
    int month{1};
    int day{1};
public:
    Date(int y, int m, int d) {
        year = y;
        month = m;
        day = d;
        std::cout << "构造 Date 对象: \t";
        print();
    }
    void print() const {
        std::cout << year << "-" << month << "-" << day << '\n';
    }
};

int main() {
    Date day(2018, 8, 18), // 定义类对象时自动调用匹配的构造函数
            day2{2019, 6, 1}; // 使用列表初始化
}
```

- 这个构造函数带有 3 个形参，在定义类对象时也必须提供对应的实参，可以采用函数调用形式（即圆括号传递实参 day(2018, 8, 18)），也可以用 {} 形式的列表初始化提供实参(day2{2019, 6, 1})。

```
int main() {  
    Date day(2018, 8, 18), // 定义类对象时自动调用匹配的构造函数  
          day2{2019, 6, 1}; // 使用列表初始化  
}
```

- 和普通的函数调用一样，如果定义类对象时少于或多于 3 个实参，则编译器会报错。

```
// Date day;    // 错：没有合适的默认构造函数可用  
// Date day(2010, 1, 2, 3); // 错：没有重载函数接受 4 个参数
```

“Date day;”的“缺少默认构造函数”的错误是因为一旦定义了自己的构造函数，编译器就不再生成默认构造函数了。

如果仍然想使用默认构造函数，怎么办？

- 可以显式地添加一个不带参数的默认构造函数：

```
Date() {}
```

也可以通过 `default` 关键字来指示编译器生成默认构造函数。即将上面的语句改为：

```
Date() = default;
```

- 下面的Date类包含了 2 个构造函数:

```
class Date {  
private:  
    int year{2000};  
    int month{1};  
    int day{1};  
public:  
    Date() = default;  
    Date(int y, int m, int d) {  
        year = y;  
        month = m;  
        day = d;  
        std::cout << "构造 Date 对象: \t";  
        print();  
    }  
    void print() const {  
        std::cout << year << "-" << month << "-" << day << '\n';  
    }  
};
```

```
class Date {
private:
    int year{2000};
    int month{1};
    int day{1};
public:
    Date() = default;
    Date(int y, int m, int d) {
        year = y;
        month = m;
        day = d;
        std::cout << "构造 Date 对象: \t";
        print();
    }
    void print() const {
        std::cout << year << "-" << month << "-" << day << '\n';
    }
};
```

```
int main() {
    Date day(2018, 8, 18), // 定义类对象时自动调用匹配的构造函数
           day2{2019, 6, 1};
    Date day; // ok, 调用默认构造函数
}
```

- 和普通函数一样，类的成员函数（包括构造函数）的参数也可以有默认值，并同样遵守默认参数一律靠右的规则：

```
class Date {  
private:  
    int year{2000};  
    int month{1};  
    int day{1};  
public:  
    Date(int y = 2000, int m = 1, int d = 1) {  
        year = y;  
        month = m;  
        day = d;  
    }  
    void print() const {  
        std::cout << year << "-" << month << "-" << day << '\n';  
    }  
};
```

- 用不同个数的实参调用了构造函数Date(int y = 2000, int m = 1, int d = 1)初始化类对象。

```
int main() {  
    Date day, day1(2011), day2{2019, 6},  
        day3{2019, 13, 8}; // 即使超范围也不会报错, 可以增加校验逻辑  
    day.print();  
    day1.print();  
    day2.print();  
    day3.print();  
}
```


- 因为这个构造函数的每个参数都有默认值，所以它就是一个默认构造函数。有了这个默认构造函数，不能在该类中再添加不带参数的默认构造函数。

```
class Date {  
private:  
    int year{2000};  
    int month{1};  
    int day{1};  
public:  
    Date() = default; // 错误：重定义默认构造函数  
    Date(int y = 2000, int m = 1, int d = 1) {  
        year = y;  
        month = m;  
        day = d;  
    }  
    void print() const {  
        std::cout << year << "-" << month << "-" << day  
    }  
};
```

8.4.2 初始化成员列表

- 在构造函数中，可以在函数体之前使用**初始化列表** (initializer list) 对类的成员变量进行初始化。

```
class Date {  
private:  
    int year{2000};  
    int month{1};  
    int day{1};  
public:  
    Date(int y = 2000, int m = 1, int d = 1)  
        : year{y}, month(m), day(d) {}  
    void print() const {  
        std::cout << year << "-" << month << "-" << day << '\n';  
    }  
};
```

- 使用初始化成员列表的好处之一是可以提高程序的执行效率。它避免了成员变量被默认初始化后又在构造函数体内重新赋值的情况，而是直接使用构造函数的参数对成员变量进行一次性初始化
- 对于 `const` 成员变量和引用类型的成员变量，必须使用初始化列表进行初始化。
- 注意：初始化成员变量的顺序并不是按照它们在初始化列表中的排列顺序，而是按照它们在类中声明的顺序进行的。

```
class Date {  
private:  
    int year{2000};  
    int month{1};  
    int day{1};  
  
public:  
    Date(int y = 2000, int m = 1, int d = 1) : day(d), month(m), year{y} {}  
    void print() const {  
        std::cout << year << "-" << month << "-" << day << '\n';  
    }  
};
```

- 虽然在初始化列表中 day 出现在最前，但实际初始化顺序仍然是 year → month → day，因为这是它们在类中声明的顺序。
- 如果成员之间存在依赖关系，初始化顺序不一致可能导致逻辑错误，编译器还可能发出警告。

8.4.3 初始化成员列表 vs 类内成员初始化器

初始化成员列表 (Initializer List)

用于在构造函数体执行前直接初始化成员变量，语法如下：

```
Date(int y = 2000, int m = 1, int d = 1) : day(d), month(m), year{y} {}
```

优点：

- 避免默认构造+再赋值，提高效率。
- 必须用于 `const` 成员、引用成员、无默认构造函数的成员或基类初始化。
- 可根据构造函数参数灵活初始化。

缺点：

- 初始化顺序由类中成员声明顺序决定，容易引起误解。
- 与构造函数体分离，可能影响可读性。

类内成员初始化器 (In-class Member Initializer)

直接在类中为成员变量指定默认值，例如：

```
int day = 1;  
int year{2025};
```

优点：

- 简洁明了，适合为成员提供统一默认值。
- 避免在多个构造函数中重复初始化逻辑。

缺点：

- 无法用于 `const` 和引用成员。
- 构造函数的初始化列表会覆盖类内默认值。
- 灵活性较低，不能根据构造函数参数差异初始化。

特性	初始化列表	类内成员初始化器
效率	高	相对较低
适用成员	所有成员，特别是 const/引用	非 const 且非引用类型成员
灵活性	高，可依构造参数变化	低，仅提供单一默认值
使用场景	精准控制初始化，必须场景多	简洁默认，适合统一初始设定

实践建议：

- `const`、引用成员必须使用初始化列表。
- 若大多数情况下成员有统一初始值，可用类内成员初始化器简化代码。
- 当构造函数有多个参数，或者需要根据参数灵活地初始化不同成员时，使用初始化列表可以使代码更直观、清晰。
- 二者可以配合使用：默认用类内初始化器，在构造函数中按需覆盖。

8.4.4 拷贝构造函数

- 定义一个类对象可以用同一个类的其他对象初始化。假如有一个类 X 的对象 x ，则可以用 x 去初始化一个新的 X 类对象：

```
X y{x};    //也可以写成 X y(x);
```

在定义了 X 的对象 y 时，用 X 的已有对象 x 对 y 进行初始化构造。

```
class Date {
private:
    int year{2000};
    int month{1};
    int day{1};
public:
    Date(int y = 2000, int m = 1, int d = 1) : year{y}, month(m), day(d) {}
    void print() const {
        std::cout << year << "-" << month << "-" << day << '\n';
    }
};

int main() {
    Date day{2018, 1, 1};
    Date day2{day};
    day.print();
    day2.print();
}
```

```
Date day2{day};
```

- Date 类中并没有以 Date 对象作为参数的构造函数，为什么 Date day2{day} 没有报告编译错误呢？
- 这是因为定义 day2 对象时调用的是一种称为 **拷贝构造函数** 的特殊构造函数。如果程序员在自定义类中没有定义**拷贝构造函数**，编译器会自动生成一个 默认拷贝构造函数。

- **默认拷贝构造函数**会按成员逐个地复制已有对象的数据成员，对于基本类型或 POD 类型（Plain Old Data），这相当于对这些成员变量的 值拷贝。这种按成员复制的方式称为 **浅拷贝**，即仅复制对象中每个成员的值，如果成员是指针或其他引用外部资源的类型，仅复制指针本身，而不会复制其所指向的资源。
- 如果一个对象需要使用一些资源，如动态内存、打开文件或网络端口，这种 浅拷贝 会导致两个对象共享同一个资源，可能会引发灾难性后果（如重复释放内存）。

- 为了验证定义类对象 day2{day}时是否调用了拷贝构造函数，可以自己定义这个拷贝构造函数并添加一些打印语句：

```
class Date {
private:
    int year{2000};
    int month{1};
    int day{1};
public:
    Date(int y = 2000, int m = 1, int d = 1) : year{y}, month(m), day(d) {}
    Date(const Date& d) : year{d.year}, month{d.month}, day{d.day} {
        std::cout << "拷贝构造函数\n";
        print();
    }
    void print() const {
        std::cout << year << "-" << month << "-" << day << '\n';
    }
};
```

执行上述程序，输出结果：

拷贝构造函数

2018-1-1

- 拷贝构造函数中的形参就是类对象的引用，能否是值参数呢？即能否将拷贝构造函数写成下面的形式？

```
X( X x);
```

```
Date(Date d) : year(d.year), month(d.month), day(d.day) {}
```

当调用这个拷贝构造函数时，需要将已有对象（比如 day）作为实参初始化给这个构造函数的形参 d，即执行 Dated{day}，这不是又调用这个拷贝构造函数了吗？如此就会无限递归下去！

- 因此，拷贝构造函数的形参至少应该是引用参数：

```
Date(Date &d) : year(d.year), month(d.month), day(d.day) {}
```

然而这个构造函数还有一个问题，就是 `const Date` 类型的对象不能作为该函数的实参（因为 `non-const` 引用形参不能用 `const` 实参初始化）。

但拷贝构造函数只是用来创建新的对象，不会修改已有的对象，为什么不将参数设置成 `const` 对象的引用(`reference to const`) 呢？

```
Date(const Date &d) : year(d.year), month(d.month), day(d.day) {}
```

8.4.5 赋值运算符：operator=

- 默认情况下，可以将一个类对象赋值给另一个类对象，如下列代码中的`day3 = day`：

```
int main() {  
    Date day{2018, 1, 1}, day3;  
    day3 = day;  
    day.print();  
    day3.print();  
}
```

通过赋值运算符 `day3 = day`，`day3` 对象复制了 `day` 对象的数据，即两者的数据成员值相同。

赋值运算符与拷贝构造函数的主要区别在于：

- **拷贝构造函数：** 用已有对象创建一个新的对象。
- **赋值运算符：** 将已有对象的值赋给另一个已经存在的对象。

那么，为什么类对象可以直接使用赋值运算符 = 呢？

这是因为对于每个类，编译器会默认生成一个赋值运算符函数，即 **operator=()** 成员函数。

对于类类型 `x`，其原型大致如下：

```
x& operator=(const x& object);
```

这个函数的参数是一个 `const` 类型的引用，返回类型是该对象自身的引用，即被赋值的对象。

day3 = day 实际上调用的是编译器为 Date 类自动生成的默认赋值运算符：

```
Date& operator=(const Date& object);
```

默认的赋值运算符函数会对每个成员变量进行逐一复制，即执行**浅拷贝**操作：仅复制各成员的值，而不考虑这些成员是否指向外部资源。这种行为对于不涉及资源管理的类来说通常是安全的。

如果类中包含了需要手动管理的资源（如动态分配的内存、文件句柄、互斥锁等），默认的浅拷贝会导致多个对象共享同一资源，可能引发严重问题（例如重复释放内存）。因此在这种情况下，程序员应当自行定义赋值运算符函数，以确保资源的正确管理。

```
class Date {
private:
    int year{2000};
    int month{1};
    int day{1};
public:
    Date(int y = 2000, int m = 1, int d = 1) : year{y}, month{m}, day{d} {}
    Date& operator=(const Date& date) {
        std::cout << "赋值运算符\n";
        year = date.year;
        month = date.month;
        day = date.day;
        return *this; // 返回被赋值对象自身的引用
    }
    void print() const {
        std::cout << year << "-" << month << "-" << day << '\n';
    }
};
```


由于赋值运算符函数返回的是被赋值对象的引用（即 ***this**），因此可以将赋值操作串联使用，如下面代码中的 `day3 = day2 = day;`。

```
int main() {  
    Date day(2018, 1, 1);  
    Date day2, day3;  
    day3 = day2 = day; // 先执行 day2 = day, 结果是 day2, 再执行 day3 = day2  
                       // day2 = day 实际是 day2.operator=(day)  
}
```

上述程序的输出是：

赋值运算符
赋值运算符

除了赋值运算符 `=`，我们还可以重载复合赋值运算符（如 `+=`、`-=`、`*=` 等），它们通常也返回当前对象的引用，以支持连续赋值或链式调用。

```
Date& operator+=(const Date& date) {  
    std::cout << "赋值加运算符\n";  
    year += date.year;  
    month += date.month;  
    day += date.day;  
    return *this; // 返回被赋值对象自身的引用  
}
```

```
int main() {  
    Date day(2018, 1, 1);  
    Date day2(1, 2, 3);  
    day2 += day;  
    day2.print();  
}
```

8.4.6 深拷贝与浅拷贝

- 在 C++ 中，对象拷贝是非常常见的操作，通常通过拷贝构造函数或赋值运算符实现。然而，如果类中包含指针成员或动态资源，默认的拷贝行为可能会导致资源管理错误。这就引出了浅拷贝与深拷贝的概念。

1. 浅拷贝 (Shallow Copy)

- **概念：** 浅拷贝指的是将源对象的所有成员变量的值直接复制到目标对象中。如果成员变量是指针，则复制的是指针的地址，而不是其指向的内容。
- **问题：** 浅拷贝会导致多个对象共享同一份资源，容易造成资源冲突、悬垂指针或重复释放的问题。

2. 深拷贝 (Deep Copy)

- **概念：** 深拷贝在复制对象时，不仅复制对象本身的数据，还会为其所持有的资源（如动态内存）分配新的空间，并复制资源内容，确保每个对象拥有独立的资源副本。
- **目的：** 防止资源共享导致的副作用，保障对象的资源独立性和内存安全。

3. 示例：String 类

- 为了更直观地理解深拷贝的重要性，我们来看一个自定义 String 类的例子：

```
#include <iostream>
```

```
#include <cstring>
```

```
class String {
```

```
private:
```

```
    char* data;
```

```
    size_t length;
```

```
public:
```

```
    // 构造函数
```

```
    String(const char* str = "") : length(std::strlen(str)) {
```

```
        data = new char[length + 1];
```

```
        std::strcpy(data, str);
```

```
        std::cout << "Constructor called\n";
```

```
    }
```

```
    // 析构函数
```

```
    ~String() {
```

```
        std::cout << "Destructor called\n";
```

```
        delete[] data;
```

```
    }
```

```
void print() const {  
    std::cout << data << std::endl;  
}
```

// 浅拷贝构造函数（不推荐）

```
/*String(const String& other) : data(other.data), length(other.length) {  
    std::cout << "Shallow copy constructor called\n";  
}*/
```

// 深拷贝构造函数（推荐）

```
String(const String& other) : length(other.length) {  
    data = new char[length + 1];  
    std::strcpy(data, other.data);  
    std::cout << "Deep copy constructor called\n";  
}
```


// 赋值运算符 (深拷贝)

```
String& operator=(const String& other) {  
    std::cout << "Assignment operator called\n";
```

// 1. 自赋值检查

```
    if (this == &other) {  
        return *this;  
    }
```

// 2. 创建新的内存副本

```
    char* newData = new char[other.length + 1];  
    std::strcpy(newData, other.data);
```

// 3. 释放旧的内存

```
    delete[] data;
```

// 4. 更新成员变量

```
    data = newData;  
    length = other.length;
```

// 5. 返回自身引用

```
    return *this;
```

```
}
```

```
};
```

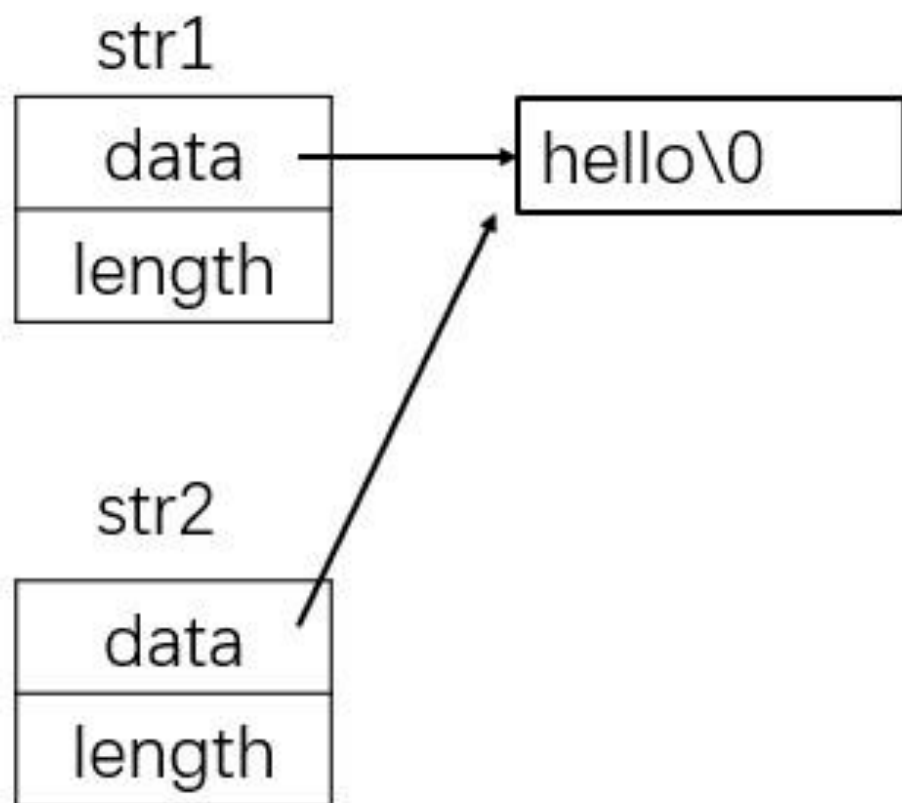
```
int main() {  
    string str1("Hello");  
    string str2 = str1; // 调用拷贝构造函数  
  
    std::cout << "str1: ";  
    str1.print();  
    std::cout << "str2: ";  
    str2.print();  
  
    string str3("world");  
    str3 = str1; // 调用赋值运算符函数  
    str2.print();  
    return 0;  
}
```

4. 浅拷贝的危害

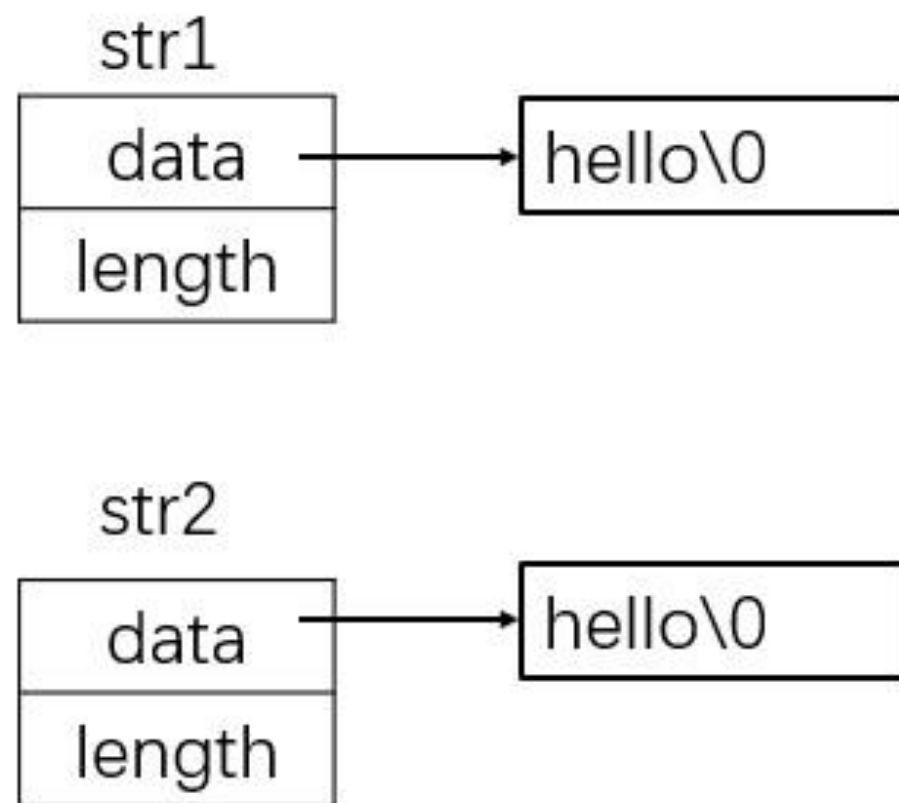
- 如果将深拷贝构造函数注释掉，改用如下的浅拷贝方式：

```
String(const String& other) : data(other.data), length(other.length) {}
```

- 那么 str1 和 str2 会共享同一块内存。当程序结束时，它们的析构函数都会试图释放这块内存，导致 **double free** 错误，程序崩溃。



(a) 浅拷贝的内存示意图



(b) 深拷贝的内存示意图

深拷贝与浅拷贝的内存示意图

5. 深拷贝的优势

- 使用深拷贝后，每个对象都有独立的资源副本。即使多个对象拷贝自同一个源，也能安全销毁各自的资源，不会发生重复释放的问题。

6. 何时需要深拷贝？

一般来说，如果类中满足以下任一条件，就应实现深拷贝：

- **拥有指针成员变量：** 指向动态内存或资源的指针。
- **负责资源管理：** 包括分配、释放、读写等资源操作。

7. 总结:

- 浅拷贝只复制指针地址，导致资源共享，存在安全隐患。
- 深拷贝复制资源内容，确保对象之间资源独立。
- 默认的拷贝构造函数和赋值运算符执行浅拷贝。
- 如类中涉及动态资源，应自定义拷贝构造函数和赋值运算符，确保安全的深拷贝语义。

8.4.7 delete

- 有时，可能希望禁止某个构造函数或赋值运算符，比如禁止编译器生成默认的拷贝构造函数或赋值运算符，可以通过delete 关键字显式地进行说明。

```
class A {  
public:  
    A(int) { /* ... */ }  
    A(double) = delete;           // 禁止使用 double 参数构造对象  
    A& operator=(const A& o) = delete; // 禁止赋值操作  
    A(const A& o) = delete;       // 禁止拷贝构造  
};
```


在 `main` 函数中尝试使用禁用的构造函数和赋值运算符会导致编译错误：

```
int main() {  
    A a(1);  
    //A a1(3.14);    // 错误: A(double) 被禁止了  
    //A a2(a);        // 错误: A(const A& o) 被禁止了  
    //a = a;          // 错误: A& operator=(const A& o) 被禁止了  
}
```

输入输出流对象也是禁止被拷贝的，因为代表输入的键盘和输出的屏幕只有一个，如果允许拷贝，就混乱了。

- delete 关键字的作用是显式地禁用某个函数，使其不能被调用。
- delete 可以用于任何函数，包括构造函数、拷贝构造函数、赋值运算符、普通成员函数等。

```
class B {  
public:  
    void func(int) { /*...*/ }           // 允许  
    void func(double) = delete;          // 禁止 double 版本  
};
```

```
class B {  
public:  
    void func(int) { /*...*/ }           // 允许  
    void func(double) = delete;          // 禁止 double 版本  
};  
  
int main(){  
    B b;  
    b.func(10);      // ✓ 正常  
    b.func(3.14);    // ✗ 编译错误! double 参数被禁止  
    return 0;  
}
```

8.4.8 隐式类型转换、explicit

- 1. 隐式类型转换
- 2. 用 explicit 禁止隐式类型转换

1. 隐式类型转换

- 带有1个参数的构造函数（或者带有多个参数，但后续参数都有默认值），定义了隐式类型转换。例如：

```
#include <iostream>
class A {
public:
    A(int x) {
        std::cout << "用 " << x << " 构造对象\n";
    }
};
```

1. 隐式类型转换

- 构造函数 `A(int x)` 被称为转换构造函数 (conversion constructor) , 因为它允许编译器在需要 `A` 类型的地方自动地将 `int` 类型的值转换成 `A` 类型对象。

```
#include <iostream>
class A {
public:
    A(int x) {
        std::cout << "用 " << x << " 构造对象\n";
    }
};
```

```
#include <iostream>
class A {
public:
    A(int x) {
        std::cout << "用 " << x << " 构造对象\n";
    }
};
```

你可以通过以下方式创建 `A` 类的对象：

```
int main() {
    A a(1), b{2}, c = 3; // 创建 A 类的 3 个对象 a, b, c
    a = 4; // 将 4 赋值给 A 类对象, 先将 4 转换为 A 类对象 A(4), 再赋值给 a
}
```

```
int main() {  
    A a(1), b{2}, c = 3; // 创建 A 类的 3 个对象 a, b, c  
    a = 4; // 将 4 赋值给 A 类对象, 先将 4 转换为 A 类对象 A(4), 再赋值给 a  
}
```

- `c = 3` 在这里是拷贝初始化, 它会调用构造函数进行隐式转换, 并非简单赋值。因此, 构造函数 `A(int x)` 实际上定义了从 `int` 到 `A` 的隐式类型转换, 可以将 `int` 类型的值转换为 `A` 类的值。
- `a = 4` 实际上相当于执行 `a = A(4)`, 即先隐式构造一个临时对象, 然后赋值。


```
void f(A a) {}  
int main() {  
    f(2); // 隐式地通过构造函数 A(int) 将 2 转换成 A 类型对象  
}
```

函数 f() 接受一个 A 类型的对象作为参数，而在调用 f(2) 时，实参是 int 类型的值 2。此时，编译器会调用 A(int x) 构造函数，将 int 类型的 2 转换为 A 类型，并用这个转换后的对象初始化形参 a。

2. 用 explicit 禁止隐式类型转换

- 对于某些类，隐式类型转换可能会带来一些问题。
- 假设我们正在编写一个函数，该函数接受一个字符串作为输入并将其转化为整数。如果没有显式的类型控制，程序可能会因为隐式类型转换产生一些难以察觉的问题。

```
#include <iostream>
#include <string>
#include <sstream>
```

```
class StringToInt {
public:
    StringToInt(const std::string& str) {
        std::stringstream ss(str);
        ss >> value;
    }

    int getValue() const {
        return value;
    }

private:
    int value;
};
```

```
// 处理函数
void processInt(StringToInt s) {
    std::cout << "Processed value: " << s.getValue() << std::endl;
}

int main() {
    processInt("123"); // 隐式调用 StringToInt 构造函数
    return 0;
}
```

假设我们不小心传递了一个无法转换为整数的字符串（例如 "abc"）。在没有显式控制的情况下，转换会失败，但失败的转换没有明确的错误提示，而是静默失败。std::stringstream 会将转换结果设置为 0，这可能导致程序产生错误的行为，并且很难发现和调试。

```
class StringToInt {  
public:  
    explicit StringToInt(const std::string& str) {  
        std::stringstream ss(str);  
        ss >> value;  
    }  
  
    int getValue() const {  
        return value;  
    }  
  
private:  
    int value;  
};
```

```
// 处理函数
void processInt(StringToInt s) {
    std::cout << "Processed value: " << s.getValue() << std::endl;
}

int main() {
    // processInt("123"); // 编译错误: 无法将 string 隐式转换为 StringToInt
    StringToInt s("123"); // 显式创建 StringToInt 对象
    processInt(s);
    return 0;
}
```

使用 `explicit` 后，我们可以确保只有在明确需要时，才会进行类型转换，从而避免隐式转换引起的意外行为。

8.4.9 类对象数组

- 对于一个没有构造函数或定义了默认构造函数的类，可以定义这种类型的对象数组。

```
class X {  
    int x{0};  
};  
int main(){  
    X arr[3]; //X有编译器生成的默认构造函数，可以定义X类的对象数组  
}
```

8.4.9 类对象数组

- 没有默认构造函数的类，则不能定义类对象的数组。

```
class X {  
    int x{0};  
public:  
    X(int i):x{i}{}  
};  
int main(){  
    X arr[3]; //X没有默认构造函数，不能定义X类的对象数组  
}
```


8.4.10 委托构造函数

- 一个类可以有多个构造函数，用于以不同方式初始化类对象。**委托构造函数**是指一个构造函数通过构造函数初始化列表调用同一个类的另一个构造函数，以减少重复代码。被调用的构造函数称为**被委托构造函数**。

```
// 被委托构造函数
Date(int y = 2000, int m = 1, int d = 1) : year(y), month(m), day(d) {
    std::cout << "Date(int, int, int) 被调用\n";
}

// 委托构造函数
Date(int* p) : Date(p[0], p[1], p[2]) {
    std::cout << "Date(int*) 被调用\n";
}
```

```
class Date {
private:
    int year{2000};
    int month{1};
    int day{1};
public:
    // 被委托构造函数
    Date(int y = 2000, int m = 1, int d = 1) : year(y), month(m), day(d) {
        std::cout << "Date(int, int, int) 被调用\n";
    }
    // 委托构造函数
    Date(int* p) : Date(p[0], p[1], p[2]) {
        std::cout << "Date(int*) 被调用\n";
    }
    void print() const { std::cout << year << "-" << month << "-" << day << '\n';
};

int main() {
    int date[]{2018, 9, 6};
    Date d(date);
    d.print();
}
```

- 1. **委托构造函数只能在初始化列表中调用被委托构造函数，不能在函数体内调用。**
- 例如，`Date(int* p) { Date(p[0], p[1], p[2]); }` 是错误的写法，因为它会创建一个新的 `Date` 对象，而不是初始化当前对象。

- 2.委托构造函数的初始化列表中不能初始化成员变量。如下代码是错误的：

```
Date(int *p) : Date(p[0], p[1], p[2]), day{20} { } // 错误!
```

- 如果需要修改 day 的值，应该将其放在函数体内，例如：

```
Date(int* p) : Date(p[0], p[1], p[2]) {  
    day = 20; // 合法，但可能不符合设计意图  
}
```

8.4.11 类体外定义成员函数和构造函数

- 类的成员函数可以在类体外定义，但必须使用**作用域解析运算符类名::**，以指明该函数属于该类，而不是普通的全局函数。

```
class Date {
    int year{2000}, month{1}, day{1};
public:
    Date(int y = 2000, int m = 1, int d = 1);
    void print() const;
};

// 类体外定义构造函数和成员函数
Date::Date(int y, int m, int d) : year(y), month(m), day(d) {}

void Date::print() const {
    std::cout << year << "-" << month << "-" << day << '\n';
}
```

```
#include <iostream>
```

```
class Date {  
    int year{2000}, month{1}, day{1};  
public:  
    Date(int y = 2000, int m = 1, int d = 1);  
    void print() const;  
};
```

```
// 类体外定义构造函数和成员函数
```

```
Date::Date(int y, int m, int d) : year(y), month(m), day(d) {}
```

```
void Date::print() const {  
    std::cout << year << "-" << month << "-" << day << '\n';  
}
```

```
int main() {  
    Date date(2021, 12, 25);  
    date.print();  
    return 0;  
}
```

- 默认参数只能在类体内声明

```
class Date {  
public:  
    Date(int y = 2000, int m = 1, int d = 1); // 仅在类声明中提供默认参数  
};  
  
Date::Date(int y, int m, int d) : year(y), month(m), day(d) {} // 这里不能再提供默认参数
```

8.4.12 头文件与源文件的分离

- 一个常见的做法是将类的声明放在头文件（如 `Date.h`）中，而将成员函数的实现放在源文件（如 `Date.cpp`）中。这样做有以下优点：
 1. 提高代码的组织性，使代码更易于维护和阅读。
 2. 避免 ODR（One Definition Rule，一次定义规则）冲突。
 3. 加快编译速度，减少不必要的重复编译。
 4. 保护代码的知识产权，如果 `Date` 类作为库提供给第三方，用户只需包含头文件 `Date.h`，

- 包含自定义头文件时，应使用双引号（`#include "xxx"`）而非尖括号（`#include <xxx>`）。

```
#include "Date.h" // 用双引号包含自定义头文件
```

练习

目标:

1. 创建 3 个文件: `Date.h`、`Date.cpp` 和 `test.cpp`。
2. 在 `Date.h` 中声明 `Date` 类和成员函数。
3. 在 `Date.cpp` 中实现 `Date` 类的成员函数。
4. 在 `test.cpp` 中包含 `Date.h` 并定义 `main()` 函数, 创建 `Date` 类的对象并调用其成员函数。
5. 确保 `Date.cpp` 和 `test.cpp` 都包含 `Date.h` 头文件。

8.5 访问控制和接口

- 设计良好的类应隐藏内部实现细节，仅暴露必要的接口，以供外部使用。这种设计思想称为**封装（Encapsulation）**，是面向对象编程的核心原则之一。封装不仅有助于保护数据不被随意篡改，还能提高代码的可维护性和可复用性。

C++中的访问控制

C++ 提供三种访问控制级别，用于限定类成员的可见性：

- `private`（私有成员）：只能被**该类的成员函数**访问，**外部无法直接访问**。
- `protected`（受保护成员）：可被**该类及其派生类**的成员函数访问，**外部仍无法访问**。
- `public`（公有成员）：对**所有代码**可见，常用于构建类的**对外接口**（Interface）。

示例：Date 类的封装设计

- Date类的year、month 和 day 都被设为 private，这可以有效防止外部代码直接篡改这些成员，保证数据的一致性和安全性。
- 但如果类完全封闭，对外部不提供任何访问方式，使用起来就毫无意义。例如，一个银行账户若不支持存取款或查询余额，就无法满足任何实际需求。
- 通过public 成员函数（如访问器 getter 和 修改器 setter）来提供安全的数据访问与修改接口：

```
int getYear() const { return year; }
```

```
void setYear(int y) { if (y > 0) year = y; }
```

```
class Date {
private:
    int year{2000};
    int month{1};
    int day{1};
public:
    Date(int y = 2000, int m = 1, int d = 1) : year(y), month(m), day(d) {}

    // 访问器 (getter) : 读取私有成员
    int getYear() const { return year; }
    int getMonth() const { return month; }
    int getDay() const { return day; }

    // 修改器 (setter) : 修改私有成员并做简单校验
    void setYear(int y) { if (y > 0) year = y; }
    void setMonth(int m) { if (m >= 1 && m <= 12) month = m; }
    void setDay(int d) { if (d >= 1 && d <= 31) day = d; }

    void print() const {
        std::cout << year << "-" << month << "-" << day << '\n';
    }
};
```

8.6 友元

- 在某些特定场景下，类可能需要允许某些外部函数或其他类访问其私有成员。为此，C++ 提供了 **友元** (`friend`) 机制，允许指定的函数或类绕过访问控制规则，直接访问类的私有和保护成员。
- 注意：友元机制是一种破坏封装性的特殊手段，应当慎重使用，仅在确有必要访问类的内部实现时使用。

8.6.1 友元函数 (Friend Function)

- 通常，外部函数无法访问类的私有成员。但如果在类的定义中使用 `friend` 关键字将某个外部函数声明为友元函数，则该函数可以访问类的私有和保护成员，尽管它本身并不是类的成员函数。

```
class Date {  
private:  
    int year{2000}, month{1}, day{1};  
    friend void print(const Date& d); // 声明外部函数 print 是 Date 类的友元  
};
```



```
#include <iostream>
using namespace std;

class Date {
private:
    int year{2000}, month{1}, day{1};
    friend void print(const Date& d); // 声明外部函数 print 是 Date 类的友元
};

void print(const Date& d) {
    // 可以直接访问 Date 类的私有成员
    cout << d.year << "-" << d.month << "-" << d.day << '\n';
}

int main() {
    Date date;
    print(date);    // 正常输出: 2000-1-1
    return 0;
}
```

```
class Date {  
private:  
    int year{2000}, month{1}, day{1};  
    friend void print(const Date& d); // 声明外部函数 print 是 Date 类的友元  
};
```

- 函数 print() 虽然是一个普通函数，但由于被 Date 声明为友元函数，它可以访问 Date 类对象的私有成员。
- 友元函数的声明必须出现在类定义中，而实现时无需再次使用 friend。
- 友元关系是单向的，即 print 可以访问 Date 的私有成员，但 Date 不能访问 print 所在类（如果有）的私有成员。

8.6.2 友元类 (Friend Class)

- 若希望类 B 能访问类 A 的私有成员，需要在类 A 的定义中将 B 声明为友元类：friend class B;。这在类之间需要紧密协作时尤为有用。

```
class A {  
    private:  
        int x;  
        friend class B; // 将 B 声明为 A 的友元类  
};
```

类A将类B声明为自己的友元类。B类对象的成员函数就可以访问A类对象的私有属性和成员函数。

```
class X {  
private:  
    int value{10};  
    friend class Y; // 声明 Y 是 X 的友元类  
};  
  
class Y {  
public:  
    void showValue(const X& obj) {  
        // 可以直接访问 X 的私有成员 value  
        cout << "X::value = " << obj.value << endl;  
    }  
};
```

```
int main() {  
    X x;  
    Y y;  
    y.showValue(x); // 输出: X::value = 10  
    return 0;  
}
```

8.6.3 友元的注意事项

- **破坏封装性：** 友元打破了类的访问控制体系，因此在设计时应保持克制，只在确有设计需要时使用。
- **单向性：** 友元关系是单向的。若 A 是 B 的友元，A 可以访问 B 的私有成员，但反之不成立。
- **不具备传递性：** 如果 A 是 B 的友元，B 是 C 的友元，并不意味着 A 是 C 的友元。
- **优先使用公共接口：** 除非无法通过普通的公有成员函数实现所需功能，应尽量避免使用友元，防止造成类之间过于紧密的耦合。

8.7 const 对象、const 成员函数 和 mutable 成员变量

8.7.1 const 对象

- 在 C++ 中，const 关键字用于修饰变量，表示其值不可更改。它可以用于内置类型，也可以用于对象、指针、引用等。

```
#include <iostream>
```

```
int main() {
```

```
    int i = 0;
```

```
    const int ci = 1;    // ci 是常量对象，不能被修改
```

```
    //ci = 3; // 错误：不能修改 const 对象（变量）
```

```
    const int* p = &i;    // p 是指向 const int 的指针
```

```
    //*p = 3; // 错误：不能通过 p 修改 p 指向的 const 对象
```

```
    const int& r = i;    // r 是一个绑定到 const int 的引用
```

```
    //r = 3;    // 错误：不能通过 r 修改 r 所引用的对象
```

```
}
```



```
#include <iostream>
```

```
class X {  
public:  
    int ival{0};  
};
```

```
int main() {  
    const X x; // x 是 const 对象  
    std::cout << x.ival << std::endl; // OK  
    //x.ival = 10; // 错误: 不能修改 const 对象 x 的成员  
  
    X y;  
    y.ival = 10; // y 不是 const 对象, 可以修改其成员  
  
    const X* p = &y; // p 是指向 const X 对象的指针  
    //p->ival = 20; // 错误: 不能通过 p 修改 p 指向的 const X 对象, 即使 y 不是 const  
  
    const X& r = y; // r 是 const X 的引用  
    //r.ival = 20; // 错误: 不能通过 r 修改 r 引用的 const X 对象, 即使 y 不是 const  
}
```

const 对象与成员函数的兼容性

- 不能通过const对象调用类的非const成员函数！

```
const Date day;  
day.setYear(2008);           // 错误：不能修改 const 对象 day  
std::cout << day.getYear(); // 错误：getYear 不会修改 day，但编译器无法确认其是否会修改数据
```

8.7.2 const 成员函数

- 在 C++ 中，为了保证成员函数不会修改对象的状态，可以将成员函数声明为 const 成员函数。这不仅能提升代码的安全性和可读性，还允许 const 对象调用这些函数。

```
int getYear() const { return year; }
```

示例：为查询函数添加 const 限定

```
class Date {  
private:  
    int year;  
    int month;  
    int day;  
public:  
    int getYear() const { return year; } // 查询函数，不修改对象状态  
    int getMonth() const { return month; }  
    int getDay() const { return day; }  
    void print() const {  
        std::cout << year << "-" << month << "-" << day << std::endl;  
    }  
  
    void setYear(int y) { if (y > 0) year = y; } // 修改成员函数  
};
```

为什么需要 const 成员函数？

当对象是 const 类型时，只能调用 const 成员函数！

```
const Date day;  
day.getYear();    // OK: getYear() 是 const 成员函数  
day.print();      // OK: print() 是 const 成员函数  
// day.setYear(2008); // 错误: setYear 不是 const 成员函数，不能被 const 对象调用
```

8.7.3 重载 const

- 在 C++ 中，成员函数中的 const 关键字属于函数签名的一部分，因此可以用于函数重载。也就是说，可以为同一个成员函数提供两个版本：一个用于非 const 对象，一个用于 const 对象，从而实现根据对象的常量性选择合适的函数。

```
class X {  
public:  
    void f() { /* 非 const 版本 */ }  
    void f() const { /* const 版本 */ }  
};
```

```
class Date {  
    int _year{2000}, _month{1}, _day{1};  
  
public:  
    int& year() { return _year; }  
    int& month() { return _month; }  
    int& day() { return _day; }  
  
    // 重载 const 函数  
    const int& year() const { return _year; }  
    const int& month() const { return _month; }  
    const int& day() const { return _day; }  
};
```

```
int main() {  
    Date day;  
    day.year() = 2008; // 调用非 const 函数: OK  
    std::cout << day.year() << std::endl; // OK  
  
    const Date day2;  
    // day2.year() = 2008; // 编译错误: 不能修改 const 对象  
    std::cout << day2.year() << std::endl; // OK: 调用 const 函数  
}
```


通过对成员函数进行 `const` 重载，可以实现根据对象的常量性选择不同的访问方式：

- 非 `const` 对象 可以调用两个版本的函数。
- `const` 对象 只能调用 `const` 成员函数。

8.7.4 mutable 成员变量

- 在 C++ 中，const 成员函数不能修改类的普通数据成员。但有一种例外：使用关键字 mutable 修饰的成员变量，即使在 const 函数中也可以被修改。

```
class X {  
private:  
    mutable int count{0}; // count 是 mutable 成员变量  
    int ival{0};  
  
public:  
    int val() const {  
        count++;          // mutable 成员总是可以被修改  
        return ival;  
    }  
};
```

适用场景

`mutable` 成员变量适用于那些**逻辑上不影响对象状态**，但**技术上需要修改数据**的场合。常见用途包括：

- **缓存机制**：缓存计算结果，提高效率。
- **访问计数器**：记录某个函数被调用的次数。
- **线程同步**：在 `const` 函数中操作互斥锁，保障并发访问的安全性。

8.8 析构函数

当一个类对象的生命周期结束时，例如：

- 它超出了其所在的作用域；
- 或者是通过 new 创建的动态对象被 delete 删除；

系统会自动调用一个特殊的成员函数——析构函数（Destructor），来执行对象销毁前的清理操作。

析构函数的定义与特性

- 一个类只能有一个析构函数。
- 析构函数的名称由类名前加上 `~` 符号构成，例如：`~X()`。
- **没有返回类型，也不能带参数**，因为它不需要由程序员手动调用，也没有返回值或传参的意义。
- 如果程序员未定义析构函数，编译器会自动生成一个默认析构函数，形式如下：

```
~X() {}
```

- 如果希望显式生成默认析构函数，可以使用 `= default`：

```
~Date() = default;
```

析构函数的作用：资源管理

如果一个类在构造函数中申请了资源（如动态内存、打开文件、建立连接等），那么就必须在析构函数中负责释放这些资源，以避免资源泄漏。这是一种基本的资源管理原则：谁申请，谁释放。

```
class IntArray {
private:
    int* data{nullptr}; // 指向动态分配的 int 数组
    int size{0};        // 数组大小

public:
    IntArray(int s) : size(s) {
        data = new int[s]; // 动态分配内存
        std::cout << "构造了一个大小为 " << s << " 的 IntArray 对象\n";
    }

    ~IntArray() {
        std::cout << "析构函数被调用\n";
        delete[] data; // 释放动态内存
    }

    void put(int i, int x) {
        if (i >= 0 && i < size) data[i] = x;
    }

    int get(int i) const {
        if (i >= 0 && i < size) return data[i];
        else return 0;
    }
};
```


8.9 内联成员函数 (Inline Member Functions)

内联函数是一种编译器优化机制，目的是减少函数调用的开销，从而提升程序的执行效率。当一个函数被定义为内联时，编译器会尝试在调用处直接插入该函数的代码，而不是进行正常的函数调用过程。

8.9 内联成员函数 (Inline Member Functions)

在 C++ 中，成员函数可以定义为内联函数，有两种常见方式：

- 1. 隐式内联：在类定义内部定义函数体
- 2. 显式内联：使用 inline 关键字在类内声明

1 隐式内联：在类定义内部定义函数体

- 当成员函数的函数体直接写在类定义内部时，该函数会被隐式地视为内联函数：

```
class X {  
public:  
    void f() {  
        // 函数体  
    }  
};
```

2. 显式内联：使用 inline 关键字在类内声明

- 也可以在类定义内部使用 inline 关键字显式声明函数为内联函数，并在类外进行定义：

```
class X {  
public:  
    inline void f(); // 显式声明为内联函数  
};  
  
void X::f() {  
    // 函数体（此处不需要再次写 inline 关键字）  
}
```

8.10 静态成员 (Static Members)

- 类的静态成员变量 (Static Member Variables) 则不同，它们属于整个类，而不是任何单个对象。所有对象共享同一个静态成员变量。
- 类的成员函数可以是非静态成员函数 (必须通过对象调用) 或静态成员函数 (可以通过类名直接调用)

静态成员变量

- 用 static 关键字声明静态成员变量。

```
class X {  
private:  
    static int count; // count 是 X 类的静态成员变量，表示 X 的对象数量  
  
public:  
    int get_count() const { return count; }  
  
    X() { count++; }      // 创建一个 X 对象时，count++  
    ~X() { count--; }     // 销毁一个 X 对象时，cot X::count{0};  
};
```

静态成员变量

- 类 X 定义中的静态成员变量 count 仅仅是声明还不是定义，因此，在类定义中不能对它初始化。如果将上述 count 变量的声明写成定义的形式：

```
//static int count{0}; // 错误：不能在类内初始化静态成员变量
```

编译器会报告如下个错误：

```
error C2864: X::count: 带有类内初始化表达式的静态数据成员 必须  
联”
```

静态成员变量

- 在哪里定义和初始化类的静态成员变量呢？ 必须在类体外定义并初始化它。

```
// 在类外定义和初始化静态成员变量  
int X::count = 0;           // 或 int X::count{0};
```

注意：

- 类体外定义静态成员变量时，其前面不能有关键字 `static`。
- 如果静态成员变量是 `const` 且为整数类型或枚举类型，那么可以在类内进行初始化 (C++11 及之后)。

C++17 及之后的 inline 静态成员变量

- 在 C++17 之前，必须在类外定义静态成员变量，显得麻烦。
C++17 允许使用 inline 关键字，使得静态变量可以在类内定义和初始化：

```
class X {  
private:  
    static inline int count{0}; // C++17: X 对象的个数  
  
public:  
    int get_count() const { return count; }  
  
    X() { count++; } // 创建了一个新的 X 对象, count 增加 1  
    ~X() { count--; } // 销毁了一个 X 对象, count 减少 1  
};
```

```
class X {  
private:  
    static inline int count{0}; // C++17: X 对象的个数  
  
public:  
    int get_count() const { return count; }  
  
    X() { count++; } // 创建了一个新的 X 对象, count 增加 1  
    ~X() { count--; } // 销毁了一个 X 对象, count 减少 1  
};
```

优点:

- 避免了必须在类外定义静态成员变量的限制。
- 保持声明和定义在同一个地方, 使代码更加整洁。
- 避免了 ODR (One Definition Rule) 违规问题。

8.10.2 静态常量

- 静态成员变量常用于在类范围内定义常量。在 C++17 及以后的版本中，可以使用 `inline static const` 关键字在类内部直接定义并初始化静态常量，而无需在类外单独提供定义：

```
class Circle {  
public:  
    static inline const double PI{3.1415926}; // C++17  
  
    Circle(double r) : radius(r) {}  
  
    double area() const { return PI * radius * radius; }  
  
private:  
    double radius{0.0};  
};
```

8.10.3 静态成员函数 (Static Member Functions)

- 与静态成员变量类似，类的成员函数也可以使用 `static` 关键字声明为静态成员函数。这种函数不依赖于任何对象实例，可以直接通过类名调用。

```
class X {  
public:  
    static inline int count{0}; // count 是类X的静态成员变量，  
                                // 表示类X的对象的个数  
public:  
    static int get_count() { return count; }; // 静态成员函数  
    X() { count++; }                        // 创建了一个新的 X 对象, count 增加 1  
    ~X() { count--; }                       // 销毁一个 X 对象, count 减少 1  
};
```

静态成员函数的特点：

- 可以在类内或类外定义，但通常在类内定义；
- 只能访问类的静态成员，不能访问非静态成员变量或函数；
- 可以通过类名调用，不依赖具体对象实例。

```
class X {
public:
    static inline int count{0}; // C++17: count 是类 X 的静态成员变量，表示类 X 的对象的个数

public:
    static int get_count() { return count; }; // 静态成员函数
    X() { count++; }                          // 创建了一个新的 X 对象,count 增加 1
    ~X() { count--; }                         // 销毁一个 X 对象,count 减少 1
};

int main() {
    X x, x2;
    X arr[3];
    std::cout << x.get_count() << '\t' << arr[1].get_count() << '\t'
               << X::get_count() << '\n';
}
```

8.10.4 类自身类型的静态成员变量

- 类中可以定义自身类型的静态成员变量，这在需要为所有对象提供统一的默认值时非常有用。例如，Date 类可以定义一个静态成员变量 `default_date`，表示默认日期。若对象在构造时未显式指定年月日，就使用这个默认值。

```
class Date {  
    // 使用 default_date 来初始化其他成员变量  
    int year { default_date.year },  
        month { default_date.month },  
        day { default_date.day };  
public:  
    // 声明一个类自身类型的静态常量成员  
    const static Date default_date;  
  
    // 构造函数默认参数使用 default_date 的值  
    Date(int y = default_date.year, int m = default_date.month, int d = default_date.day)  
        : year(y), month(m), day(d) { }  
  
    void print() const { std::cout << year << "-" << month << "-" << day << '\n'; }  
};  
  
// 类体外定义和初始化静态常量成员变量  
const Date Date::default_date { 2010, 1, 1 };
```


注意事项:

- 类自身类型在类体内是不完整类型，因此**不能**在类内使用 `inline` 关键字直接定义和初始化这类静态成员。
- 类自身类型的静态成员必须在**类外**定义和初始化。
- 如果将其声明为 `const`，该成员就不可修改；若希望可修改，则不要加 `const`。

8.11 实战：线性表及应用

关注

Youtube频道: <https://www.youtube.com/c/hwdong>

博客: <https://hwdong-net.github.io>

Udemy课程: <https://www.udemy.com/user/hwd/>